

Protein Sequence Generation and Selection for Balanced Novelty and Structural Confidence

by

Shahriar Khan Shirso
22201441

Suprio Biswas Mugdha
22299069

Anika Anis Trishna
22201146

Sinka Siddiki Aishi
22301670

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
February 2026.

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Students Full Name & Signature:



Shahriar Khan Shirso
22201441



Suprio Biswas Mugdha
22299069



Anika Anis Trishna
22201146



Sinka Siddiki Aishi
22301670

Approval

The thesis titled Protein Sequence Generation and Selection for Balanced Novelty and Structural Confidence submitted by

1. Shahriar Khan Shirso (22201441)
2. Suprio Biswas Mugdha (22299069)
3. Anika Anis Trishna (22201146)
4. Sinka Siddiki Aishi (22301670)

of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Spring, 2026.

Examining Committee:

Supervisor:
(Member)

Dr. Amitabha Chakrabarty

Professor
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)



Dr. Swakkhar Shatabda
Professor
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

Proteins carry out most of the work of the cell, and their functions are very much dependent on the sequence of amino acids of which they are composed. The computational design of new protein sequences is thus a key problem in bioinformatics with generative models emerging as a major approach to the problem. However, a useful generated sequence must satisfy two criteria: it should fold into a stable structure and it must be novel rather than being a close match to a sequence that is already found in existing databases. The objectives are in conflict as steering a model towards sequences it confidently generates increases the chance that its outputs would resemble the sequences it was trained on. The majority of systems that can effectively accomplish this trade-off use large clusters of many GPUs, but, how much of this balance can be achieved with the hardware available to an individual researcher is unclear. In this thesis, a latent diffusion model is reimplemented and extended for generating protein sequences within the representation space of a frozen protein language model, combining this with a scoring stage that filters candidates before reporting. All of the training, generation and evaluation is conducted on one 24 GB consumer graphics card. The outcomes are most significant as a result of selection. On the curated PDB subset, Mean pLDDT scores improve from 40.63 to 72.89, on the full structural archive it goes from 48.06 to 71.49 and on the deduplicated SwissProt corpus it goes from 52.17 to 71.73, compared to the reference scores for real sequences of 78.87 and 80.7, respectively. These improvements indicate a difference that is more than just quantitative, because confidence gained through longer training comes at the expense of novelty, whereas confidence gained by filtering does not incur the same trade-off. The target pLDDT of 70 is reached after only 225 epochs of training the SwissProt from which a selection is made for this pLDDT, giving a cluster diversity of 0.9920 which is essentially unchanged and a novelty of 54%. However, the cluster diversity decreases substantially for extended training on the redundant structural corpus. The findings, combined with the ablations, imply that, under a limited computational budget, more improvements can be achieved by investing in data quality and post-generation selection rather than increasing the size of the model.

Keywords: Protein sequence generation; Protein design; Diffusion models; Protein language models; Sequence novelty; Protein structure prediction; pLDDT; Representation learning; Computational protein design; Protein generation

Acknowledgement

First of all, we would like to express our heartfelt gratitude to Almighty Allah for giving us the strength and perseverance to work on this thesis. We would also like to express our deepest gratitude to our parents for their unconditional love, support, and encouragement, which have been a constant source of strength during the course of this thesis. We thank Dr.Amitabha Chakrabarty,from our heart for his valuable contribution,guidance and necessary feedbacks alongside his continuous encouragement guiding us throughout,thus improving quality of our research. In addition, we are deeply grateful to our research assistant Mr. Azwad Aziz, for his dedicated support with experiments and guidance, which greatly facilitated the progress of this study. Finally, we also acknowledge ourselves for the perseverance, dedication, and effort invested in this thesis, together these contributions made this work possible and rewarding and we are proud of what has been achieved.

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	ix
List of Tables	x
Nomenclature	xi
1 Introduction	1
1.1 Background	1
1.2 Rationale of the Study or Motivation	2
1.3 Problem Statement	2
1.4 Objective	3
1.5 Methodology in Brief	4
1.5.1 Data Collection and Preprocessing	4
1.5.2 Latent Encoding	5
1.5.3 Model Architecture	5
1.5.4 Training Objective	6
1.5.5 Sequence Generation	6
1.5.6 Scoring and Selection	7
1.5.7 Structural Evaluation and Metrics	7
1.6 Scopes and Challenges	8
2 Literature Review	10
2.1 Preliminaries	10
2.2 Problem Discussion: From Variant Prediction to Foldable Sequence Generation	12
2.3 Review of Existing Research	14
2.3.1 Variational Autoencoders (VAEs) for Protein Sequence Gen- eration	14
2.3.2 Diffusion Models for Protein Sequence and Structure Generation	15
2.3.3 Latent Diffusion Models for Protein Sequence Generation . . .	16

2.3.4	Protein Language Models (PLMs) for Direct Sequence Generation and Evaluation	16
2.3.5	Hybrid Generative Models (Structure + Sequence, PLM + Diffusion, Diffusion + Prior-Guided Design)	17
2.3.6	Deep Generative Models: Computational Heaviness and Scalability	18
2.4	Key Findings	23
2.5	Research Gap	24
3	Requirements, Impacts and Constraints	26
3.1	Final Specifications and System Requirements	26
3.2	Functional Requirements Framework	26
3.3	Derived Hardware and Architecture Specifications	28
3.4	Residue Representation and Data Contracts	28
3.5	Dataset Partitioning Standards	29
3.6	Target Metrics and Empirical Achievements	29
3.6.1	Mean Structural Confidence (pLDDT)	29
3.6.2	Sequence Diversity (Cluster Distance at 0.5)	29
3.6.3	Sequence Novelty (L1 Distance to Training Set)	29
3.6.4	Fréchet Sequence Distance	30
3.7	Societal Impact and Accessibility	30
3.8	Ethical Issues, Governance, and Environmental Standards	31
3.9	Comprehensive Risk Management Framework	31
3.9.1	Technical Operational Risks and Fallback Mechanisms	31
3.9.2	Methodological Risks and Safeguards	32
4	Proposed Methodology	33
4.1	Design Process or Methodology Overview	33
4.2	Preliminary Design or Design (Model) Specification	36
4.2.1	Frozen Encoder	36
4.2.2	Latent Normalisation	36
4.2.3	Denoiser	37
4.2.4	Sequence Decoder	37
4.2.5	Diffusion Formulation	37
4.3	Data Collection:	38
4.3.1	Data Cleaning	39
4.3.2	Data Transformation	42
4.3.3	Data Integration	42
4.3.4	Data Reduction	42
4.3.5	Summary of Preprocessed Data	44
4.4	Implementation of Selected Design	45
4.4.1	Training Objective	45
4.4.2	Proxy Foldability Objective	48
4.4.3	Bio Composition Objectives	50
4.4.4	Generation	52
4.4.5	Selection Stage	52
4.4.6	Evaluation Implementation	56
4.4.7	Code and Data Availability	56

5	Result Analysis	57
5.1	Performance Evaluation	57
5.1.1	Experimental Phase (PDB 2000–2015)	57
5.1.2	Whole PDB Dataset (1971–2015)	58
5.1.3	SwissProt 106k Corpus	58
5.1.4	Unselected Generation	59
5.2	Analysis of Design Solutions	59
5.2.1	Encoder Capacity and Family	59
5.2.2	Training Duration	59
5.2.3	Noise Schedule	60
5.2.4	Proxy Foldability Objective	60
5.2.5	Bio Composition Objectives	60
5.3	Final Design Adjustments	60
5.4	Statistical Analysis	61
5.4.1	Experimental Phase (PDB 2000–2015)	61
5.4.2	Whole PDB Dataset (1971–2015)	61
5.4.3	SwissProt 106k Corpus	62
5.4.4	Master Consolidated Analysis and Scaling Relationships	62
5.5	Comparisons and Relationships	63
5.6	Discussions	64
5.6.1	Data Redundancy and Diversity Collapse	64
5.6.2	Mechanics of Proxy Objectives	64
5.6.3	System Bottlenecks and Headroom	65
5.6.4	Methodological Limitations	65
6	Conclusion	67
6.1	Summary of Key Findings	67
6.2	Contributions to the Field	70
6.3	Future Directions	71
	Bibliography	72

List of Figures

1.1	Overview of the proposed protein sequence generation pipeline.	4
1.2	Overview of the proposed protein sequence generation pipeline.	5
2.1	Illustration of the protein structure	13
2.2	ProteinVAE architecture: Encoder: Attention mechanism with fixed query vector ([CLS] vector) is used to extract bottleneck representation z from pretrained language model hidden representation H . Decoder: Deconvolutional networks are used for upsampling. Attention modules are applied on both dimensions.	14
2.3	Overall architecture of the TaxDiff.	15
2.4	Protein engineering leverages autoregressive, VAE, GAN, and diffusion sequence models to generate and predict functional protein variants.	18
4.1	Preprocessing attrition profile for the Whole PDB Dataset (1971–2015), showing sequence retention at each cleaning stage	41
4.2	Preprocessing attrition profile for the Experimental Phase (PDB 2000–2015), showing sequence retention at each cleaning stage.	41
4.3	Preprocessing attrition profile for the SwissProt 106k corpus, highlighting the Min-Hash redundancy bucketing stage.	42
4.4	Training corpus repetition Rep(train) across the three datasets (evaluated on fixed samples of 2,000 sequences across $n \in \{8, 16, 32, 64\}$).	45
4.5	Model training process.	45
4.6	Sequence generation after model training.	52
4.7	Scoring process.	55

List of Tables

2.1	Performance Summary of Multiple Methods and Their Limitations . .	19
4.1	Inventory of framework configurations and ablations investigated across the three experimental phases.	35
4.2	Signal retention levels $\bar{\alpha}_t$ across timesteps for Tangent ($d = 10$) and Cosine ($s = 0.008$) schedules.	38
4.3	Stage-by-stage preprocessing attrition for the two structural data configurations.	40
4.4	Stage-by-stage preprocessing attrition for the SwissProt 106k corpus.	40
4.5	Summary of preprocessed dataset characteristics across all three data phases.	44
5.1	Statistical analysis of all measured configurations in the experimental phase (PDB 2000–2015).	61
5.2	Statistical analysis of the whole PDB dataset (1971–2015) configurations.	62
5.3	Statistical analysis of the SwissProt 106k corpus configurations. . . .	62
5.4	Cross-model benchmark comparison against published sequence generation baselines.	63

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

ACE2 Angiotensin-Converting Enzyme 2 receptor

CNN Convolutional Neural Network

ELBO Evidence Lower Bound

GAN Generative Adversarial Network

LSTM Long Short-Term Memory network

MSA Multiple Sequence Alignment

NOS Diffusion-Optimized Sampling

PLM Protein Language Model

PLM + Diffusion Hybrid framework combining Protein Language Models with Diffusion Models

RNN Recurrent Neural Network

VAE Variational Autoencoder

VQ-VAE Vector Quantized Variational Autoencoder

Chapter 1

Introduction

1.1 Background

Proteins are vital components of all cells and carry out a wide range of functions in cells such as catalysis, signalling, immunity, and transport. The function of a protein is determined by its amino acid sequence that defines the three dimensional fold the chain assumes. The design of novel sequences with desired properties thus represents a long-standing research objective in biotechnology, bioinformatics and drug discovery.

The approach to this problem has been changed by machine learning. Protein language models, variational autoencoders, and diffusion models can all be trained to learn regularities in protein sequence from large protein databases, and then generate novel and similar sequences for therapeutic protein and enzyme design. These methods are only feasible in practice because of two bottlenecks. The first one is computational. A number of recent generative models involve large neural networks and long training periods, but are limited to environments with extensive resources. Most published results can't be reproduced or extended by a researcher with a single consumer graphics card, and the gap is increasing with larger models.

The second thing is biological plausibility. Generative models produce sequences that are novel by construction, but this alone is not sufficient. A model can emit a series that is not in the database, and can not be folded into something. A model may release a sequence that is not present in the database, and does not fold into anything. As experimental and computational validation is expensive, therefore a generator that produces many implausible candidates shifts cost downstream rather than removing it.

Both of these restrictions are intertwined and affect this work. A model can be cheaper, by either training fewer instances or training smaller instances, but each of these approaches tends to result in a model that is less plausible. The question is not simply how to build a smaller version, but where along that trade-off a usable operating point exists, and which components are actually responsible for moving it.

One approach suggests latent-space generative modelling, where an amino acid can be sampled from the continuous representations generated by a pretrained protein

language model, instead of directly generating the amino acids, and a decoder used to generate a sequence from the continuous representations. The number of parameters to optimize is reduced significantly, as the representation is not trained, but used as a fixed feature extractor, and the structural knowledge learned in the representation is preserved. Then, at the expense of discarding some of the output, post-generation filtering can further improve the quality of the delivered set.

1.2 Rationale of the Study or Motivation

Protein sequence generation has been shown to have definite utility in drug design, enzyme engineering and synthetic biology in recent years. Methods that work best, however, require large amounts of computing, and the literature is missing the decision-making that went into the choice of single methods. A reader who reads the final system with its result will find out what the result is, but not which part of the system was responsible for the result.

This is essential for those who are working under a constraint. Replacing a part that contributes less with a less expensive alternative costs nothing. If it is load-bearing, removing it breaks the system. Without an ablation there is no way to tell the two apart, and a limited compute budget is spent on the wrong things. This study thus has two purposes. It creates a latent diffusion model for generating protein sequences that can be trained and executed on a single graphics card, and quantifies the influence of the parts of the model that are relevant. The ablations involve the frozen capacity/size and model family of the encoder, the diffusion noise schedule, the training time, two auxiliary training objectives to enhance the foldability and the post-generation selection stage.

In addition, three training corpora are compared, which vary in size and redundancy. This wasn't in the original plan. It was needed because if we had been able to get a good structural run with a very low diversity, then we would have known that was an issue in the training data and not the model. It altered the nature of the work, following the path of what they could do as a compute constraint as opposed to altering their architecture.

1.3 Problem Statement

Generating sets of protein sequences that are both structurally plausible and diverse, while having limited compute, is difficult, and current methods are not able to identify which of their components is responsible. In this thesis, the problem is divided into three parts.

Quality under constraint : Training and evaluation of a latent diffusion framework are done end-to-end with just one consumer graphics card, and sequences generated using this framework have a high level of structural confidence, not limited to non-random sequences only.

Attribution : The contribution of each component should be assessed and not assumed, and then the computation should be spent where it makes a difference in the result. This involves keeping all other settings constant while changing one setting, and then the configuration of the training needs to remain consistent between runs.

Honest reporting of the quality–diversity trade-off : Structural confidence, diversity, and novelty can all be enhanced at the cost of the others, and a single headline number can be achieved at the expense of properties which a reader would be interested in. The performance or effectiveness of the framework should be assessed across the three, and any improvement in one should be accompanied by a report of the cost in the other two.

1.4 Objective

In this work, we propose a framework for generating biologically plausible and foldable protein sequences using latent diffusion with PLM representations of the frozen protein language, which is also computationally efficient, and analyze the factors that influence the performance. Specific objectives are as follows:

1. Construct three training corpora of varying size and redundancy; measure the cost of each preprocessing choice in terms of number of sequences kept. The corpora includes a representative sample of the structure, the total structural archive and an identity redundant sequence database.
2. Train a latent diffusion model with one consumer graphics card that contains a frozen protein language model encoder, a transformer denoiser on the normalised latents per-residue, and a learned sequence decoder which maps denoised latents to amino acids.
3. Report the statistics of the four measures of the quality of the sequences generated on four separate axes; predicted structural confidence, distributional similarity to natural proteins, and sequence diversity, novelty with respect to the data used to train the model, but not the best of those.
4. Ablate framework components to determine what each contributes: encoder capacity over a nineteen-fold range, another family of encoders, the diffusion noise schedule, the length of training, and post-generation selection stage.
5. Conduct a test to determine if the selection process can be shifted into training. There were two auxiliary objectives that were pursued for this purpose, one based on a learned confidence predictor, and one based on differentiable composition statistics, both of which are reported along with their results.
6. Isolate the effect of generation from that of filtering; select models that are trained as modified and then use the same trained models to correctly report on the quality of the contribution that is made, using the same modified models for the selected subset as well as for the unfiltered output.

1.5 Methodology in Brief

There are five stages in the pipeline: data collection and preprocessing, latent encoding, diffusion training, generation with length conditioning, and post-generation scoring and structural evaluation. A general description of the pipeline is provided in Figure 1.1.

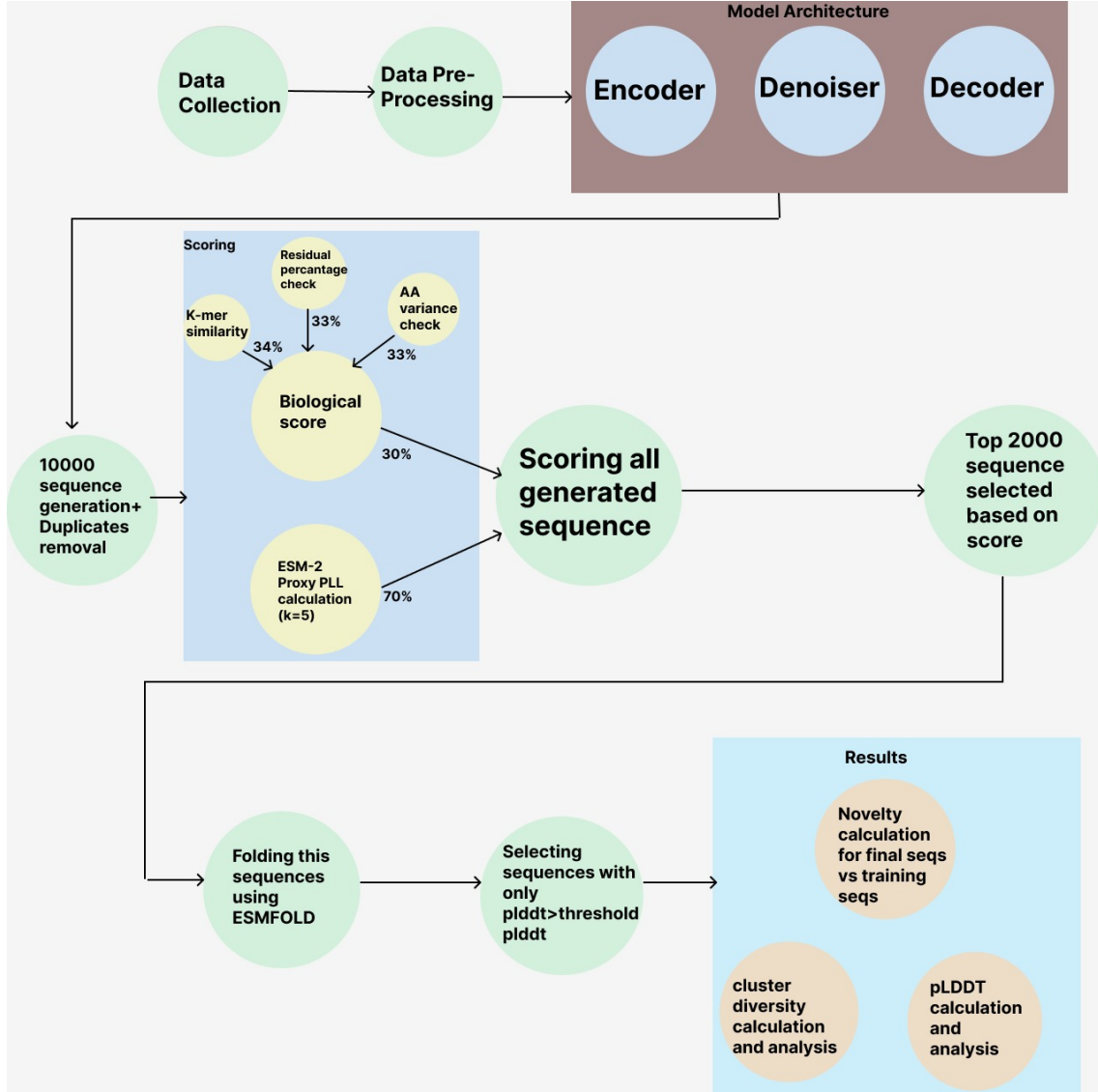


Figure 1.1: Overview of the proposed protein sequence generation pipeline.

1.5.1 Data Collection and Preprocessing

Three corpora were created. The first is a subset of the structures, carefully selected from the years 2000–2015 and restricted to sequences of 128–256 residues. This resulted in 18,349 training structures, with a further 4,525 structures held out for testing from the years 2016–2018.

The second corpus consists of the complete structural archive from 1971–2015, restricted to sequences of 40–512 residues, yielding 51,829 training sequences.

The third corpus is a reviewed sequence database clustered at 50% sequence identity, with one sequence retained per cluster, and further filtered to sequences of 128–256 residues. This resulted in 105,991 training sequences and 1,176 sequences belonging to held-out clusters.

Sequences containing non-canonical residues were not repaired, and sequences outside the specified length ranges were discarded rather than truncated. Truncation was avoided because cutting a chain through the middle of a domain can expose its hydrophobic core to the solvent. Training on such truncated structures could therefore encourage the model to generate sequences containing exposed hydrophobic cores that are unlikely to fold correctly.

1.5.2 Latent Encoding

The cleaned sequences are passed once through a frozen ESM-2 encoder and stored as per-residue latent tensors. The default encoder is the 35M-parameter ESM-2 variant, which produces latent representations with a width of 480.

The diffusion process, denoiser, and decoder operate entirely in the normalised latent space. This space is obtained by standardising the latent representations across real residue positions. The encoder is never fine-tuned; consequently, gradients are propagated only through the denoiser and decoder during training.

1.5.3 Model Architecture

The architecture consists of three principal modules, as shown in Figure 1.2.

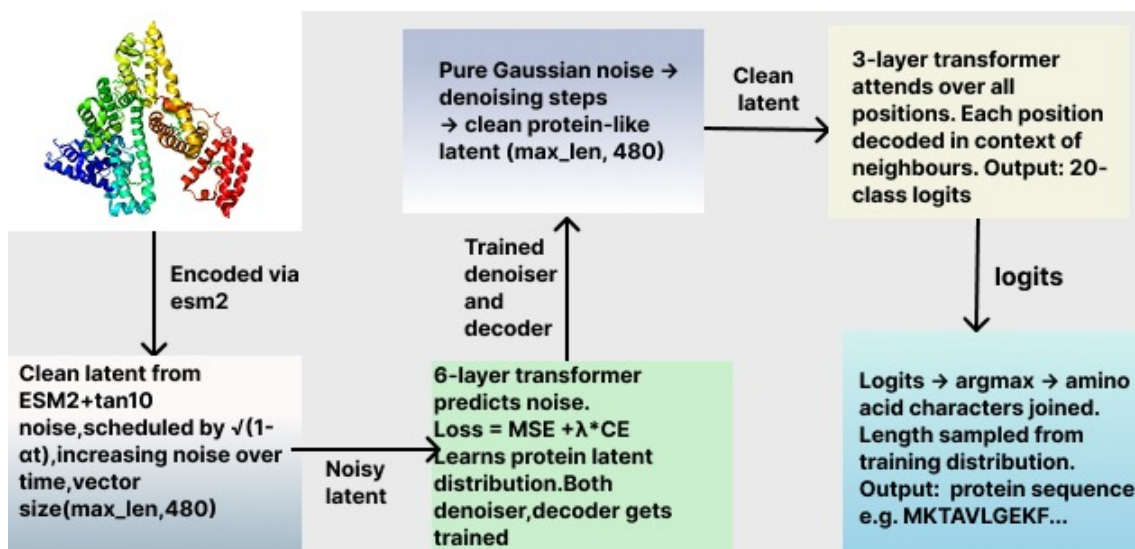


Figure 1.2: Overview of the proposed protein sequence generation pipeline.

Diffusion Denoiser

The denoiser is a six-layer pre-normalisation Transformer with a hidden width of 512, eight attention heads, and learned positional encodings. Rather than directly

predicting the noise added during the forward diffusion process, the denoiser predicts the clean latent representation. The forward process corrupts the latent representations with Gaussian noise according to a variance schedule with a steepness parameter of 10 over 1,000 diffusion timesteps.

Self-conditioning is applied with a probability of 0.5. Thus, half of the training steps receive the corresponding provisional clean-latent estimate as an additional input channel, while the remaining training steps receive zeros in this channel. This ensures that the unconditional path used during the first sampling step is also adequately represented during training.

Sequence Decoder

The decoder is a standalone three-layer Transformer with a width of 480. It receives complete normalised latent sequences together with a padding mask and produces logits over exactly 20 canonical amino acid classes. The decoder does not reuse the output head of the protein language model. Instead, it is a separately learned module trained specifically to map latent representations to amino acid sequences.

1.5.4 Training Objective

The training objective combines a masked mean squared error (MSE) term for the denoiser with a cross-entropy term for the sequence decoder. The decoder loss is weighted by 0.2. The overall objective can be expressed as

$$\mathcal{L} = \mathcal{L}_{\text{diffusion}} + 0.2\mathcal{L}_{\text{decoder}}, \quad (1.1)$$

where $\mathcal{L}_{\text{diffusion}}$ denotes the masked MSE between the predicted and clean latent representations, and $\mathcal{L}_{\text{decoder}}$ denotes the amino acid cross-entropy loss.

The decoder is trained using two inputs: the clean latent representation and the denoiser’s own denoised latent representation, with the latter gradient-detached. The contribution of the decoder loss is additionally weighted according to the signal level, since the decoder is primarily required to accurately reconstruct sequences from the low-noise latent representations encountered near the end of sampling.

The two loss components are separated such that the decoder objective does not propagate gradients through the denoiser output.

AdamW is used for optimisation with a learning rate of 10^{-4} , weight decay of 10^{-5} , gradient clipping at 2.0, and a batch size of 32. Training is performed for 180 epochs. The learning rate is linearly warmed up and subsequently annealed using cosine scheduling, with warm restarts every 15 epochs over 500 warm-up steps. For generation, an exponential moving average of the denoiser weights is maintained with a decay rate of 0.9999.

1.5.5 Sequence Generation

During generation, the sequence length is first sampled from the empirical length distribution of the corresponding training corpus. The sampled length is then used

to construct the attention mask during the reverse diffusion process. Consequently, no residues are generated in padded positions, which are removed after sampling. Denoising is performed using deterministic DDIM sampling with 250 fixed steps over the requested diffusion timesteps. The sampling timesteps are uniformly distributed. Once denoising is complete, the final latent representation is passed through the sequence decoder, and the amino acid with the maximum predicted probability is selected independently at each residue position.

For every experimental run, 10,000 candidate sequences are generated.

1.5.6 Scoring and Selection

Each generated candidate is assigned a composite plausibility score without explicitly folding the sequence. The score consists of a statistical heuristic component weighted at 30% and a protein language model pseudo-log-likelihood component weighted at 70%.

The heuristic component is calculated from three statistics measured relative to the training corpus: three-mer usage, polar residue fraction, and amino acid composition entropy. The overall score can therefore be represented as

$$S_{\text{total}} = 0.3S_{\text{heuristic}} + 0.7S_{\text{PLM}}, \quad (1.2)$$

where $S_{\text{heuristic}}$ represents the aggregated statistical score and S_{PLM} represents the protein language model pseudo-log-likelihood.

The top 2,000 candidates according to the composite score are retained for subsequent evaluation. A clustering-based stratification strategy was also implemented and evaluated but was not included in the final design for the reasons discussed in Chapter 5.

1.5.7 Structural Evaluation and Metrics

The selected sequences are evaluated using mean predicted Local Distance Difference Test (pLDDT) scores and structural predictions generated with ESMFold.

Four primary metrics are reported together:

1. **Structural quality:** mean pLDDT, measuring the predicted confidence of the generated structures.
2. **Distributional similarity:** Fréchet distance computed over pooled sequence embeddings, measuring the similarity between generated and natural sequence distributions.
3. **Sequence diversity:** clustering density measured at 50% and 95% sequence identity thresholds.
4. **Sequence novelty:** normalised edit-distance novelty measured against the training data.

In addition to these four primary metrics, the number of repeated n -mers is reported as an additional measure of sequence redundancy.

Since the generation and selection stages are treated as independent components, each trained model is evaluated both on the selected subset of generated sequences and on the corresponding unfiltered output. This allows the contribution of the generation process to be distinguished from the effect of post-generation filtering.

1.6 Scopes and Challenges

There are a number of difficulties that arise when generating protein sequences under computational constraints during this work.

Training Data Redundancy : It is difficult to obtain a training set that is both well-distributed and diverse. Protein databases are unevenly distributed because proteins are not deposited uniformly across sequence space, but rather according to research interest, resulting in well-studied folds being represented many times. A model trained on such a corpus learns this concentrated distribution and consequently reproduces it. This was the most important contributor to the low output diversity observed in this work and was addressed through identity-based deduplication rather than by modifying the model.

Compute Cost and Model Scale : Larger protein language models and diffusion models are computationally complex and expensive to train, restricting their use to high-resource settings. Conversely, smaller independently trained models may lack the biological signal required for successful protein generation. The compromise adopted in this work is to keep the representation model fixed while learning the structural knowledge through a comparatively small number of trainable parameters.

Sampling Step Count : Diffusion sampling involves a trade-off in the number of denoising steps. Increasing the number of steps increases the computational cost of generation, whereas using fewer steps may compromise the quality of the generated sequences. The noise schedule can also be easily misinterpreted. Under a steep schedule, uniformly spacing the sampling steps in timestep index places most steps in a region where the latent representation changes very little, while allocating comparatively few steps to the region where the latent representation develops meaningful structure.

Distinguishing Diversity from Quality : Structural confidence and sequence diversity can move in opposite directions, allowing a model to improve one property while compromising the other. Evaluating a model using only a single parameter can therefore provide a misleading assessment of its performance. Consequently, this work considers four complementary measures to evaluate the generated sequences.

Separating Generation from Filtering : When the quality of the final set is improved through post-generation filtering, the improvement should not be attributed

solely to the model responsible for generation. To distinguish the contributions of generation and filtering, the same trained models are evaluated twice: once using the filtered set and once using the unfiltered generated output.

Proxy Objectives Are Not the Quantity They Approximate : Two approaches were investigated for optimising structural confidence during training using learned or differentiable proxy objectives. Neither approach produced a measurable benefit. Both are nevertheless reported because their results provide useful information about the limitations of such proxy objectives and the considerations required when designing objectives that approximate structural properties.

Practical Constraints : Long training runs on a single machine were occasionally interrupted by disk exhaustion and power failures. Consequently, checkpointing was used to allow training and evaluation to resume from the appropriate epoch and per-sequence folding stage. Structure prediction represents the most significant evaluation cost, requiring approximately 2.7 seconds per sequence, which limits the number of sequences that can be folded within a single experiment.

Chapter 2

Literature Review

2.1 Preliminaries

Some knowledge of molecular biology, bioinformatics and machine learning would be helpful for understanding this work, as it is at the cross section of these three fields.

1. Protein and Amino Acid Sequences

Proteins are basic biological molecules composed of amino acids linked together by peptide bonds. The structure, stability, and function of a protein is determined by the order of the amino acids. Proteins are usually represented in a set of 20 standard letters, each having a different single letter code (e.g., Alanine = A, Glycine = G, Tryptophan = W). The combination and arrangement of these 20 symbols produce the vast array of biological forms and functions found in nature.

2. Foldability

Foldability is the ability of a given amino acid sequence to reliably fold into a stable and well-defined 3-D form, rather than staying disordered or misfolded. It is the key quality criterion of any generated sequence as a sequence with no plausible fold will not be able to perform a biological function, no matter how 'natural-looking' it is at the sequence level. In most cases foldability is estimated *in silico* with structure-prediction tools rather than through wet-lab structural biology, which makes rapid, automated confidence metrics crucial to screen large collections of generated candidates in structural biology.

3. Novel Protein Sequence Generation

This refers to the computational design of sequences of amino acids that are not found in nature, or even in the training database, as opposed to simply extracting or slightly mutating established sequences. The aim is to sample new areas of sequence space, while maintaining the statistical and physico-chemical regularities that allow natural proteins to be folded into functional shapes. These regularities can be learned directly from large databases of sequences and structures, instead of relying on handcrafted rules, resulting in the application of generative machine learning models, such as autoregressive language models, VAEs, GANs, and diffusion models, for this task.

4. **Bioinformatics**

Generative and predictive models are constructed and tested on a computational scaffold which is provided by bioinformatics, complemented by sequence databases, alignment tools and structure repositories. The classical approach of relating a new sequence back to known biology is sequence alignment and homology search.

5. **Machine Learning on Biological Sequence Modelling**

Machine learning can uncover complex and non-obvious patterns in large biological datasets that are difficult to capture using hand-crafted features. In particular, deep learning learns hierarchies of representations directly from raw sequence data and these representations can then be utilized for designing new sequences, predicting structure, or scoring how natural a candidate sequence is.

6. **Protein Language Models (PLMs)**

Protein Language Models are deep learning models that are trained on a vast collection of protein sequences. They learn the statistical relationship between amino acid residues by simply observing the co-occurrence frequencies from millions of evolutionarily related sequences, and use this information to interpret sequences as sentences in a biological language. Without any hand-engineered evolutionary features, Transformer-based PLMs such as the ESM family have been used to predict structure, function, and mutational effects, and more increasingly, to generate or score novel sequences.

7. **Variational Autoencoders (VAEs)**

Variational Autoencoders are probabilistic generative networks that encode biological sequences to a compressed continuous latent space and decode samples from that space to get back biological sequences. VAEs can create novel sequence variants that adhere to the statistical distribution of a training set, and the latent space can be explored to understand the directionality of the latent space to biological meaningful variants.

8. **Diffusion Models**

Diffusion models are generative models which are trained to reverse a gradual noising process: a clean data point (a sequence, a structure or a learned embedding of either) is corrupted in a step by step with noise during training, and the model learns the denoising process step by step during its generation time. In the case of proteins, diffusion can be applied to discrete amino acid tokens or to continuous PLM embeddings (latent diffusion) and/or to 3D coordinates and sequence.

9. **Evaluation Metrics: pLDDT, PLL, TM-score and BLAST**

Four metrics recur throughout the literature on generated protein sequences and are used repeatedly in this thesis. **pLDDT** (predicted Local Distance Difference Test) is a per-residue confidence score, originally introduced alongside AlphaFold, that structure-prediction tools such as AlphaFold2 and ESMFold output to indicate how confident the model is in the local structure it predicts; and can be averaged over a sequence to provide a single foldability proxy.

PLL (pseudo-log-likelihood), sometimes reported as pseudo-perplexity, measures how natural a sequence looks to a pretrained PLM by masking residues one at a time and scoring the probability the model assigns back to the true residue; it correlates with fitness and, in several recent studies, with structural confidence, making it a cheap proxy for foldability that does not require running a full structure predictor. **TM-score** (Template Modeling score) quantifies the global structural similarity between two protein structures (or between a predicted structure and a reference) on a scale that is largely independent of protein length, and is the standard way to check whether a generated sequence folds into the intended topology or how self-consistent a design pipeline is. **BLAST** (Basic Local Alignment Search Tool) is the classical sequence-alignment algorithm used to search a query sequence against a database of known sequences; in the generative-modeling literature it (or PLM-based successors to it) is used to confirm that generated sequences are genuinely novel rather than near-duplicates of training data.

2.2 Problem Discussion: From Variant Prediction to Foldable Sequence Generation

Earlier fragments of this line of research have been approached as a *prediction* problem; that is, given an observed history of protein or virus sequences, predict what mutations or variants are likely to occur next. This is helpful for applications like surveillance, but it is not the problem this thesis addresses. The work here focuses on *generation*: the generation of entirely new sequences of amino acids that are structurally foldable and therefore biologically plausible, but not necessarily from any one evolutionary branch or pathogen.

This change in perspective alters the set of previous works that are relevant. Variant forecasting with phylogenetic tree, epidemiological staging or lineage-specific fine-tuning is only tangentially useful here, as it is tailored to investigate a dynamic population of known sequences but not to investigate uncharted sequence space. Instead, the growing number of works in the field of *de novo* protein sequence generation, where the central open problem lies the same as this thesis targets: How to obtain a statistically novel sequence with high foldability? State-of-the-art generative architectures (large PLMs, diffusion models over structure or sequence, and hybrid PLM-diffusion systems) typically require either large models, or large compute budgets, or both. These sections briefly summarize this literature, and pose two questions of each method: What is the quality of the resulting sequences (quantified by pLDDT, PLL, TM-score, novelty and other statistics), and how much computation is needed to achieve that (model size, number of GPUs/TPUs, training time). This dual lens directly feeds into the research gap that this thesis addresses: A lightweight generation pipeline which is designed to include post-generation filtering with the foldability in mind right from the beginning.

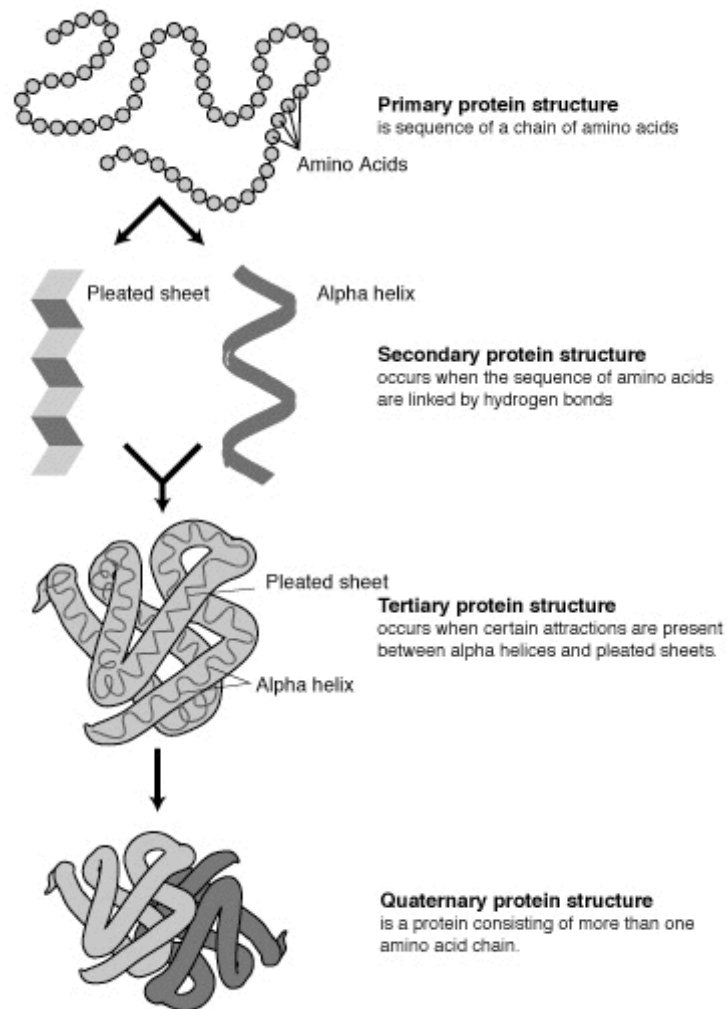


Figure 2.1: Illustration of the protein structure
[1]

2.3 Review of Existing Research

2.3.1 Variational Autoencoders (VAEs) for Protein Sequence Generation

VAEs were one of the first deep generative architectures used for protein sequences, and are still a useful, relatively low-complexity baseline. A more recent advancement is the fully Bayesian VAE developed by Riesselman et al. [2] which trains directly from multiple sequence alignments to learn site-independent and pairwise evolutionary constraints, which are then used to predict mutations impacts in the latent space, and can be trained on a single GPU in hours instead of days, which is exactly why it is used later in this thesis as a non diffusion performance floor. Hawkins-Hooker et al. [3] trained conditional VAEs on families of luciferase proteins (luxA) to generate functional variants, and were capable of generating variants with modestly sized VAEs, even without the use of a multi-GPU cluster (just tens of thousands of homologous sequences as training data). Lyu et al. [4] proposed ProteinVAE which combines an attention-based encoder that pools a pretrained language-model representation into a small latent space with a deconvolutional decoder to generate highly diverse sequences without incurring high inference costs, but as the review literature in this field [5] points out, naive decoding from such latent spaces can produce sequences that are diverse but not natural in particular or foldable in particular, exactly the structural ceiling that this thesis later empirically observes.

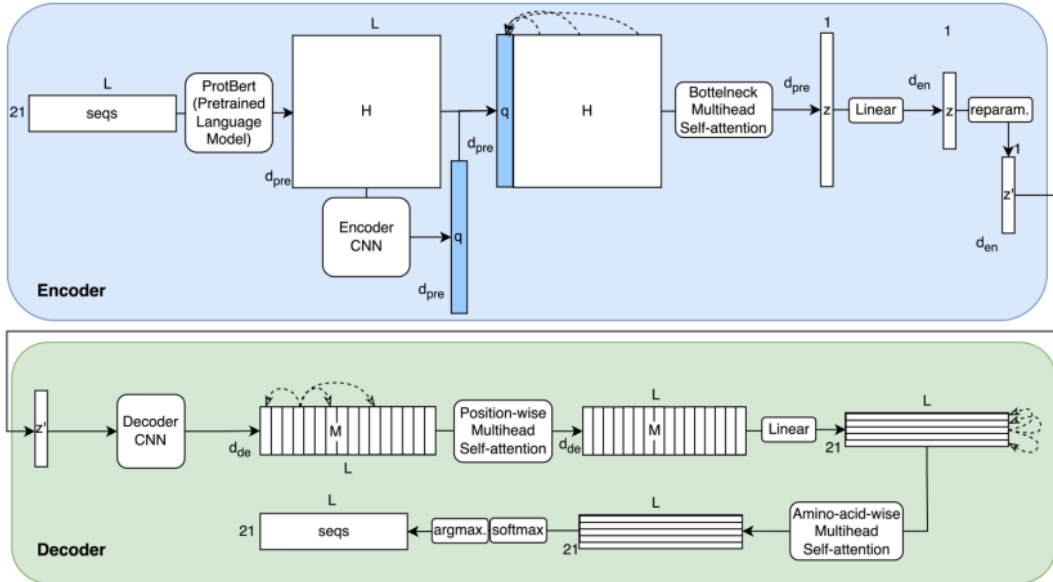


Figure 2.2: ProteinVAE architecture: Encoder: Attention mechanism with fixed query vector ([CLS] vector) is used to extract bottleneck representation z from pretrained language model hidden representation H . Decoder: Deconvolutional networks are used for upsampling. Attention modules are applied on both dimensions.

[4]

In this type of application, VAEs are always the least expensive of the generative models in terms of the computational expenses (single consumer GPU, hours of training) and evidence surveyed in this thesis shows that they do not possess the strong inductive bias needed to reliably pass high foldability thresholds.

2.3.2 Diffusion Models for Protein Sequence and Structure Generation

The main reason diffusion models are the new paradigm for the generation of foldable sequences and structures is that they have a much stronger structural inductive bias than a single pass VAE decoder, as they perform iterative denoising. The joint structure-sequence model [6] of Anand Achim is an equivariant denoising diffusion model, where the backbone and full-atom coordinates are trained using rotation/translation-equivariant diffusion while the sequence is being learned using masked diffusion; training such a joint model is complex and requires multi-GPU training on curated structural data. TaxDiff [7] instead performs diffusion purely in sequence space, guided by taxonomic labels sourced from NCBI across more than 23,000 categories, and reports a mean pLDDT of 39.68 with a 98.3% structural pass rate when trained on UniRef50 (protein length capped below 256 residues); the original authors used eight NVIDIA RTX 4090 GPUs, whereas the streamlined re-implementation used later in this thesis converges in roughly three hours on a single RTX 4070, demonstrating substantial headroom for compute reduction after the architecture is simplified.

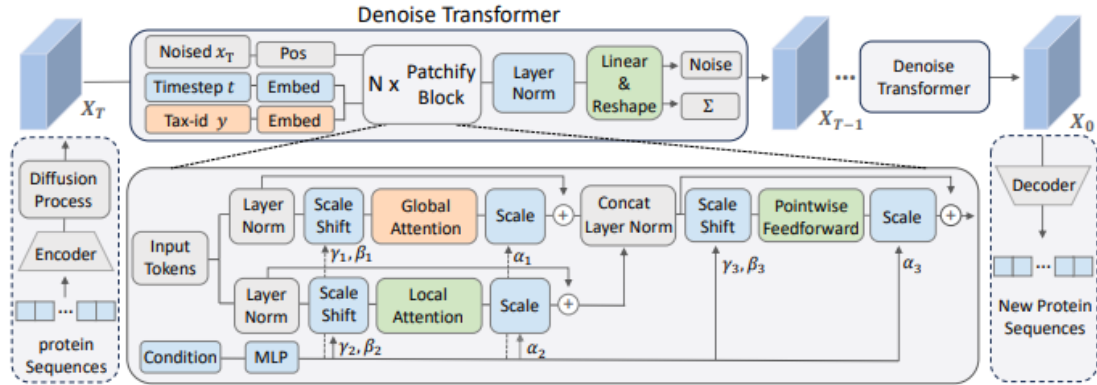


Figure 2.3: Overall architecture of the TaxDiff.

[7]

ProteinSGM is proposed by Lee, Kim and Kim [8] who show that protein backbone generation can be reformulated as an image-inpainting problem for a score-based (diffusion) generative model of 2D representations of the structure and validate designs using TM-score against predicted structures and circular-dichroism spectroscopy of synthesized proteins, the image-style U-Net architecture again being comparatively heavy to train from scratch. At the end of structural realism spectrum, RFdiffusion [9] operates on top of the large pretrained RoseTTAFold structure-prediction network, fine-tuning it as a denoiser to generate a protein backbone for any arbitrary functional scaffold, achieving experimentally validated binders and

enzymes, but at the cost of being built on top of a large structure network already trained with a cluster of GPUs, which makes it far out of single-GPU reach. Chroma [10] takes diffusion a step further, incorporating a graph-neural-network backbone that is conditioned to natural-protein statistics, and conditioning a diffusion process with a programmable, correlated diffusion process, with both training and sampling requiring significant GPU infrastructure. The collection of papers reveals a consistent trend: Each additional architectural change, whole-atom equivariance, structure-network fine-tuning, and complex-scale generation introduces additional GPU cost, making the search for lighter-weight diffusion variants like the sequence-only, latent-space variants reviewed next even more essential.

2.3.3 Latent Diffusion Models for Protein Sequence Generation

The primary mechanism that the literature employs to scale diffusion’s structural inductive bias to more modest compute budgets is operating over a compressed latent representation instead of raw discrete (or 3D) tokens. The work in this thesis follows this approach, as well as the others that the literature uses to scale down diffusion’s structural inductive bias to more modest compute budgets, by operating over a compressed latent representation instead of raw discrete (or 3D) tokens. Meshchaninov et al. [11] The original implementation of DiMA using four NVIDIA A100 (80GB) GPUs to achieve a mean pLDDT between 75.71 and 75.74 on ESM-2 35M-scale sequences, the same starting point this thesis’ re-implementation attempted to compress. Quinn et al. [12] proposed Latent Space Diffusion (LSD), which demonstrated that diffusion combined with the protein latent representation from an Autoencoder outperforms masked language models trained on UniRef50 embeddings across generation and discriminative downstream tasks. Quinn et al. note, however, that the latent representations learnt from the diffusion step may be less general in nature than that of the discriminative PLM they are distilled from. Hallee et al. [13] used a pretrained PLM and a diffusion process on its representations, they also reported an improvement in biological plausibility compared to a diffusion-only baseline, albeit at the price of the time and computational resources needed to train and backpropagate through a large pretrained PLM. Lisanza et al. [14] took this a step further, creating a hybrid of RoseTTAFold size and a RoseTTAFold pseudo-perplexity scored by ESM along with AlphaFold2 pLDDT for validating multistate and functional designs, important methodologically here, as it is one of the few large-scale systems to use a PLL-style pseudo-perplexity score, which this thesis uses for cheap pre-folding filtering. All these latent and hybrid-space diffusion methods collectively validate that much of the foldability benefits of a complete discrete diffusion model can be preserved when operating in an embedded compact space, the core architectural assumption of the framework developed in this thesis.

2.3.4 Protein Language Models (PLMs) for Direct Sequence Generation and Evaluation

Another closely related field of use is the use of PLMs as generators and evaluators, not only as encoders within a diffusion pipeline. Ferruz, Schmidt and Höcker [15]

trained ProtGPT2, in 128 NVIDIA A100 GPUs on UniRef50 for four days, is a 738-million-parameter autoregressive transformer (GPT-2 architecture) that was tested on sequences generated by it using AlphaFold-derived pLDDT (37% generated sequences had pLDDT > 70) and remote-homology search against natural sequences to confirm novelty, illustrating both what is achievable with large-scale PLM generation and how far outside typical university-lab compute budgets such training runs sit. Madani et al. [16] scaled this idea with ProGen, a 1.2-billion-parameter conditional transformer trained on 256 Google Cloud TPU v3 cores for 2 weeks on roughly 280 million sequences, and continued to scale up and experimentally demonstrated novel proteins across a variety of families, this time in the scope of the TPU-pod. Lin et al. [17] introduced ESMFold, While the largest PLM (up to 15 billion parameters) can be used to derive structure prediction directly from the model, this thesis employs only the frozen, much smaller ESM-2 35M encoder together with the openly available ESMFold structure head, which is used as the structural-validation tool of this thesis.

Finally, Liu et al. [18] proposed PLMSearch which is a sequence search based on PLM embedding, more sensitive in detecting remote homology than classical sequence search tools like BLAST [19] and still fast enough for large-scale novelty screening for newly generated sequences, providing a modern, PLM-native alternative to BLAST-like novelty checking of generated sequences. Combined, this subsection demonstrates a clear scaling trend, and the best PLM generators obtain more than a billion parameters and demand GPUs or TPUs clusters running for days to weeks, underscoring the need for the frozen, small-encoder approach used later in this thesis.

2.3.5 Hybrid Generative Models (Structure + Sequence, PLM + Diffusion, Diffusion + Prior-Guided Design)

Some recent models have taken the best of the above and compressed them into a single pipeline that includes structural priors, PLM representations, and diffusion sampling. Dauparas et al. [20] proposed a comparatively lightweight message passing neural network for designing sequences given a fixed backbone structure, called ProteinMPNN. It is much less expensive to train and run than above diffusion-based structure generators, and is often used downstream of structure-generating tools like RFDiffusion for filling in sequences for a provided backbone structure. Krapp et al. [21] also implements a context-aware geometric deep learning network for sequence design, based on the local structural context, but preferring to stick to a lighter graph-based architecture than a complete generative diffusion stack. On the PLM-diffusion side, Liu et al. [22] introduced his work on ProteinDT, a multimodal framework that integrates a PLM-derived representation space with a diffusion-based decoder for enabling the design and editing of proteins guided by natural language descriptions, demonstrating that natural-language conditioning can be stacked on top of the same latent-diffusion machinery reviewed above with no proportional rise in the cost of generation time. Ren, He and Huang [23] present ProteinRG, a pipeline consisting of three stages, fine-tuning the ESM-2 encoder, diffusing over the representations, and using a conditional Wasserstein GAN decoder with gradient penalty To tackle this issue of data scarcity, they validate generated

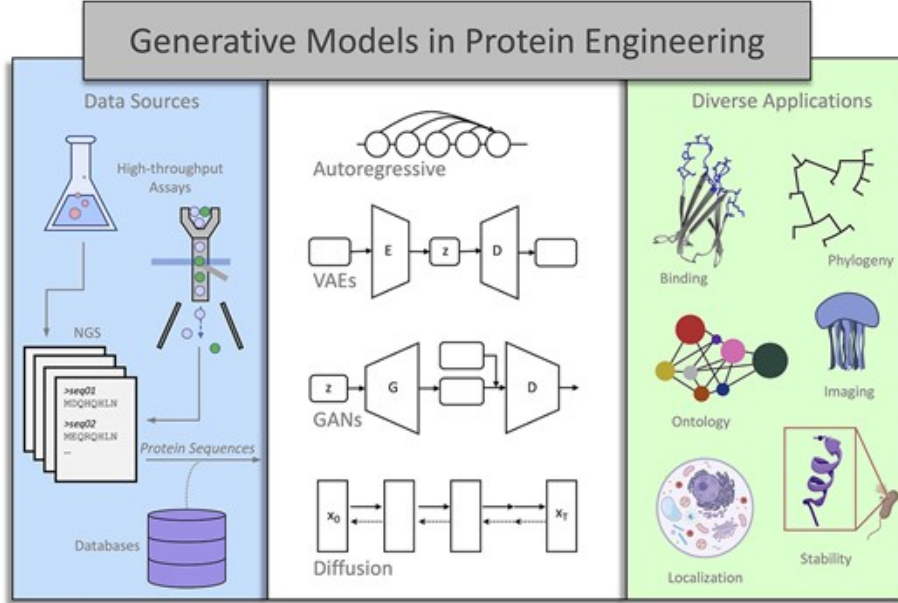


Figure 2.4: Protein engineering leverages autoregressive, VAE, GAN, and diffusion sequence models to generate and predict functional protein variants.

[5]

sequences using both pLDDT and TM-score, and report sequences reaching about 95% global identity with natural sequences under conditional generation, among the closest published analogues to the combined PLM-encoder-plus-diffusion-plus-scoring strategy used in this thesis.

Bai et al. [24] used a learned structural mask-prior and a denoising diffusion process to enhance the quality of the protein inverse folding task, demonstrating that it is possible to add a cheap and informative prior into the diffusion pipeline to improve the quality without just scale the network. In this subsection, the recurring theme is that hybrid designs are much more about the clever insertion of a lower cost module, a fixed backbone, a fixed PLM encoder, a lightweight prior, wherever a higher cost module, a backbone, a PLM encoder, a heavy prior would be needed, and is the design philosophy followed by the framework of this thesis as well.

2.3.6 Deep Generative Models: Computational Heaviness and Scalability

When looking at the models as a whole, it is evident that there is a consistent scaling issue. Review articles on generative protein design [5], [25], [26] and on efficient generative modelling of sequences more broadly [1], [27], [28], [29], [30] converge on the same set of burdens. First, parameter counts have increased from tens of millions to more than a billion (ProGen at 1.2B parameters, ProtGPT2 at 738M parameters, ESMFold’s language model at up to 15B parameters). Second, the hardware needed to obtain state-of-the-art results is beyond the scope of a typical single-GPU research setup: ProtGPT2 needed 128 A100 GPUs for 4 days; ProGen needed 256 TPU v3 cores for 2 weeks, and the original DiMA required four A100 GPUs for 10 days, while TaxDiff used eight RTX 4090 GPUs. , respectively. Third, the iterative denoising

process of diffusion is inherently expensive: each iteration adds multiple inference costs, and models trained on large, pretrained networks (e.g., RFDiffusion, Chroma) also incur training costs for their diffusion models in addition to the pretrained networks. The fourth cost comes from the integration overhead: Hybrid PLM-diffusion systems like ProteinDT and ProteinRG require back-propagation through the large pretrained PLM or at least inference with the pretrained PLM on each training step, which is costly even if the PLM itself is not fine-tuned. On this backdrop, it is worthwhile noticing that the lightweight, single-consumer-GPU alternatives surveyed here (ProteinMPNN’s fixed-backbone design, TaxDiff and DiMA’s compact transformer denoisers, and DeepSequence’s small VAE) show that competitive foldability scores do not strictly depend on cluster-scale compute, but rather on a pipeline designed around a frozen, small encoder and an aggressive post-generation filtering strategy. This is where the framework that this thesis constructs fits into.

Table 2.1: Performance Summary of Multiple Methods and Their Limitations

Reference	Methodology	Results/ Key Findings	Dataset	Key Limitations
[31]	Generated influenza protein variants capturing realistic mutations.	Median Levenshtein distance: 2.00.	6,840 influenza H3N2 sequences (NCBI IVR).	Mode collapse; high false positives/negatives; data-hungry.
[29]	Generated novel proteins with preserved structural motifs.	High structural reconstruction fidelity.	Protein Data Bank (PDB).	Mostly in silico; limited generalization; complex dependencies.
[32]	EVE predicts pathogenicity across large-scale human variants.	Predictions for ~36M variants in 3,219 genes.	MSAs, ClinVar, DMS datasets.	Context-specific effects missed; weak for rare proteins.
Continued on next page				

Table 2.1 – continued from previous page

Reference	Methodology	Results/ Key Findings	Dataset	Key Limitations
[3]	Generated functional luxA variants with improved properties.	Luminescence and solubility retained.	69,130 luciferase homologs (InterPro).	Long-range dependency and alignment issues.
[33]	Introduced Argmax Flows and Multinomial Diffusion.	Competitive non-autoregressive performance.	Text and image segmentation datasets.	Complex training; weaker than autoregressive models.
[25]	Accelerated protein generation using generative models.	Faster than traditional pipelines.	Protein sequence and structure databases.	Lab validation required; functionality not guaranteed.
[27]	arDCA effective with limited training data.	Outperforms DeepSequence on small datasets.	Pfam and viral protein families.	MSA-dependent; weaker on rich datasets.
[26]	Generative models accelerate protein sequence design.	Faster sequence generation.	Protein sequence databases.	Lab validation needed; dataset bias issues.
[6]	Generated physically realistic protein structures.	Biophysically plausible backbones.	CATH 4.2 S95 (PDB).	No evolutionary or viral modeling.
[34]	VQ-VAE-F improved image generation coherence.	Best-performing variant reported.	Image generation datasets.	Image-only; high computational cost.

Continued on next page

Table 2.1 – continued from previous page

Reference	Methodology	Results/ Key Findings	Dataset	Key Limitations
[35]	Improved discrete latent diffusion for images.	FID 6.81 on ImageNet.	FFHQ, LSUN, ImageNet.	Not applicable to biological sequences.
[36]	VAE generated spike variants with reduced immunogenicity.	Smooth immunogenicity variation.	SARS-CoV-2 spike sequences.	No wet-lab validation; indirect immune modeling.
[37]	Efficient high-resolution latent diffusion.	State-of-the-art image synthesis.	ImageNet and large image datasets.	Image-specific; autoencoder dependent.
[28]	Accelerated peptide therapeutic design.	Promising peptide candidates.	Peptide datasets.	Model selection unclear; data scarcity.
[38]	Generated target-specific molecules from sequences.	Favorable docking scores.	ChEMBL protein–drug pairs.	Ligand-focused; no protein evolution modeling.
[39]	Designed proteins with improved expression and binding.	High expression and affinity.	Protein design benchmarks.	Limited biological validation.
[4]	Generated diverse protein sequences.	High sequence diversity.	Protein sequence databases.	Many irrelevant sequences.
[5]	Compared VAEs, GANs, and diffusion models.	Diffusion most stable and diverse.	Protein sequence databases.	Validation and biological priors remain difficult.

Continued on next page

Table 2.1 – continued from previous page

Reference	Methodology	Results/ Key Findings	Dataset	Key Limitations
[40]	Predicted SARS-CoV-2 antigenic evolution.	Correct variant ranking and immune escape.	GISAID, DMS, outbreak.info.	Sampling bias; partial fitness modeling.
[30]	Predicted frequent mutations using DL constraints.	High-frequency mutation prediction.	GISAID pandemic sequences.	Limited to historical constraints.
[1]	Generated protein backbones with motif fidelity.	Accurate backbone reconstruction.	Protein Data Bank.	Mostly in silico; sampling inefficiency.
[41]	Predicted Omicron variants before emergence.	Wet-lab validated infectivity and escape.	Omicron Spike S1 sequences.	Limited to Spike S1/RBD.
[42]	LSTM modeled viral gene sequences.	85% accuracy at 20 epochs.	HBV and HCV sequences.	Long-range dependency limitations.
[7]	Taxonomic diffusion improved protein design.	Best TM-score and RMSD.	UniRef50 (<256 aa).	Taxonomy-dependent; functional nuance loss.
[13]	Masking diffusion improved protein generation.	Strong generation performance.	Protein sequence datasets.	Computationally expensive; PLM-dependent.
Continued on next page				

Table 2.1 – continued from previous page

Reference	Methodology	Results/ Key Findings	Dataset	Key Limitations
[43]	Reviewed generative models for viral evolution.	Review study.	Multiple viral databases.	No new experimental results.
[44]	Generated realistic SARS-CoV-2 mutations.	Biologically plausible mutations.	SARS-CoV-2 sequences.	No wet-lab validation.
[11]	Latent diffusion improved protein sequence quality.	High-quality, diverse outputs.	Protein sequence datasets.	Below reference metrics; PLM-dependent.
[12]	LSD models improved discriminative prediction.	Outperformed MLMs.	UniRef50.	Weaker representation learning.
[45]	Improved diffusion quality–diversity balance.	Better long-range dependencies.	Protein sequence datasets.	High computational complexity.
[46]	Generated realistic RNA sequences.	Bit scores comparable to natural RNA.	Rfam and RNA mutant datasets.	CFG and structure prediction sensitivity.

2.4 Key Findings

The current literature on generative protein sequence design shows high computational and resource costs. This has become a barrier for research groups with limited resources. Five common problems can be seen in the reviewed studies.

The first problem is model size and architecture. Many models now have hundreds of millions or even billions of parameters. As the model size increases, more com-

putation is needed to reproduce the reported results. It becomes even harder to extend these models. The second problem is hardware and infrastructure. Many state-of-the-art methods use multiple GPUs or TPU pods. Most academic labs and student research teams do not have access to such hardware. Having the expensive hardware even via rent isn't usually economically feasible. The third problem comes from diffusion itself. Diffusion models need many denoising steps. This increases the computational cost during inference. Sample quality is also closely related to the number of reverse-diffusion steps. Therefore, the cost remains high even after training. We have also seen a fourth problem in hybrid models. Pre-trained models are quite expensive, adding generative pipeline adds on even more computation and requires more hardware. The fifth problem is scaling in sequential and autoregressive methods. Some methods work well on small benchmarks. Their performance can decrease when they are applied to larger and more realistic problems.

Overall, the reviewed studies show a strong focus on increasing model scale to improve sequence quality. Foldability and the cost in terms of novelty are reported less often.

There is also some counter-evidence. Methods such as ProteinMPNN, TaxDiff and DeepSequence achieve competitive results with much lower computational costs. Some use lightweight architectures. Others use structural information. These results suggest that a more efficient, filtering-driven pipeline can be a possible alternative to using a larger generative model. This is the possibility tested in the present work.

2.5 Research Gap

Current protein sequence generation models need high computational resources. Many use billion-parameter language models. Some need multiple GPUs for diffusion training. Others fine-tune large pretrained structure models. These requirements make them difficult to run on a single machine.

Another issue is foldability. Most existing methods check foldability after generation. They do not use foldability during the main generation process. There is no foldability-aware filtering stage in many of these pipelines. The whole generated pool is sent to structure prediction. This can waste computation. Some poor candidates could be removed earlier using a cheaper signal. There is another important issue. Generated sequences need to satisfy two requirements. They should fold into stable structures. They should also be new. They should not be close relatives of sequences in the training data.

These two goals can conflict. Longer training can improve structural confidence. However, it can also make the model closer to the training data. The same can happen when the model is guided toward highly likely sequences. This can reduce novelty. Many studies focus more on one of these goals. The other goal is often discussed less. Because of this, it is still unclear how much foldability a lightweight system can achieve. Also, there is no clear indication when novelty begins to diminish.

The implications of this trade-off are also not well understood. This leaves room for a light-weight generation framework. Such a system ought to Develop sequences with fewer computations. It needs to be able to filter candidate before. However, such expensive structure prediction is not feasible. The filtering should be done using a good approximation to folding potential. The framework should also indicate novel and foldability.

It would be better if the following filtering signal could be transferred to the training process. This will enable the model to learn the structure confidence directly. It could reduce the need for post-generation selection.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and System Requirements

The main engineering challenge of this work is to build a fully functional and high fidelity protein sequence generation system that can perform full end-to-end training, latent representation caching, sampling generation and post-hoc structural folding within the physical constraints of a single consumer graphical processing unit (GPU). This system was deployed in a local host workstation on a single NVIDIA RTX 4090 GPU, with 24GB of Video Random Access Memory (VRAM), on a standard CPU architecture and Anaconda Python runtime environment, whereas the workflow for protein design is typically deployed on large state of the art industrial compute clusters or multi-node distributed infrastructure. This goal is accomplished by separating out the software and hardware requirements into tightly coupled functional, algorithmic and data specification contracts which are structurally plausible and sequence viable.

3.2 Functional Requirements Framework

Overall system functionality is described by nine functional requirements, each of which establishes a verifiable operational rule intended to preserve computational feasibility and biological validity.

Compute Capping and Hardware Isolation

All aspects of the model, including data preprocessing, latent extraction, diffusion training, candidate sampling, and post-hoc structural prediction, are required to operate on a single-GPU system. The required system RAM depends on the dataset being processed. For the SwissProt corpus used in this work, approximately 25.5 GB of RAM was required for allocation. Therefore, a system with 32 GB of RAM is considered sufficient for the complete workflow. The CPU architecture does not require any multi-GPU dependency or cloud-based acceleration cluster.

Latent Compression and Memory Management

A pre-trained, frozen Protein Language Model (PLM) encoder projects raw amino

acid sequences into a continuous latent space during sequence encoding. Because repeatedly passing sequences through the encoder during diffusion training introduces substantial computational overhead, the continuous sequence representations are stored directly on local storage using disk memory mapping. This eliminates the need to repeatedly encode the same sequences during training.

Generative Latent Diffusion Architecture

The denoising architecture is based on a generative latent diffusion framework. The denoiser is implemented as a lightweight Transformer that performs denoising within the continuous latent manifold. The network contains approximately 22 million parameters, allowing efficient denoising sweeps under different noise schedules while remaining suitable for consumer hardware.

Discrete Sequence Decoding

A dedicated lightweight sequence decoder is used to map denoised continuous latent vectors into discrete amino acid probability distributions for canonical sequence positions. The decoder consists of a three-layer Transformer with self-attention mechanisms.

Continuous Noise Schedule Control

The diffusion framework uses hyperbolic-tangent noise schedules as well as cosine noise schedules, as mathematically configurable continuous-time noise schedules, in order to control the information destruction/reconstruction dynamics over time steps

$$t \in [0, 1]. \tag{3.1}$$

Sequence Space Filtering and Quality Control

Raw FASTA data will be automatically filtered with sequence length limits (L=40-512 residues, 128-256 residue) and non-standard/ambiguous residues (B, J, O, U, X, Z) being discarded.

Corpus Deduplication and Diversity Regulation

The preprocessing engine uses sequence-identity clustering and deduplication routines (via MinHash hashing) to reduce the redundancy in the training set and avoid generative collapse in addition to enforcing structural diversity between the output manifolds.

Automated Candidate Selection

Candidate selection is performed after sequence generation by either a language model log-likelihood or pseudo-log-likelihood (pLL) scoring of raw sequence generated batches, which enable selection of the best candidates prior to structural folding.

Post-Hoc Structural Validation

Structural candidates of selected sequences are automatically identified using ESM-Fold, which allows to estimate the level of structural confidence, (expected Local

Distance Difference Test, pLDDT), and the global foldability without actually synthesizing them physically in a wet lab experiment

3.3 Derived Hardware and Architecture Specifications

The functional requirements are modularized into hardware and software components. The frozen PLM encoder, ESM-2, transforms a set of discrete sequence strings of length L into continuous embeddings of dimension D , where D ranges from 320 to 1024 depending on the selected encoder. During latent generation, memory usage remains comparatively stable because all encoder weights are frozen, avoiding the additional VRAM requirements associated with gradient backpropagation through the encoder.

The denoiser Transformer contains six attention layers, eight attention heads, and a hidden dimension of 512, with approximately 22.4 million parameters. This relatively modest model size enables batch sizes of up to 128 latent vectors for diffusion training, making efficient use of the tensor cores of the RTX 4090 while remaining within its available memory. The discrete decoder uses a compact three-layer Transformer architecture designed to reconstruct amino acid token probabilities, with approximately 8.68M parameters. Through PyTorch tensor files, disk memory mapping can be enabled for pre-computed latent caches of sequences up to $L = 512$ residues. These cached representations are stored on standard NVMe solid-state storage and accessed with low latency during GPU mini-batch training.

3.4 Residue Representation and Data Contracts

Data-ingestion contracts precisely define the conversion of biological sequence strings into the mathematical inputs required by the model. Only the 20 canonical amino acids are permitted:

$$\mathcal{A} = \{A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y\}. \quad (3.2)$$

These correspond to Alanine (A), Cysteine (C), Aspartate (D), Glutamate (E), Phenylalanine (F), Glycine (G), Histidine (H), Isoleucine (I), Lysine (K), Leucine (L), Methionine (M), Asparagine (N), Proline (P), Glutamine (Q), Arginine (R), Serine (S), Threonine (T), Valine (V), Tryptophan (W), and Tyrosine (Y).

Any non-canonical or ambiguous single-letter code, including B, J, O, U, X, and Z, is removed during ingestion to maintain strict output dimensional consistency.

In addition, background residue-frequency checks are performed to ensure that the data splits retain natural proteomic distributions. Common residues, such as Leucine at approximately 8.5–9.8% and Alanine at approximately 7.2–8.4%, are monitored to ensure consistency with natural proteomic distributions across training and evaluation sets.

3.5 Dataset Partitioning Standards

To measure generative generalization in a rigorous way, the training, validation and testing sets are divided into two different sets for training. The first one is based on temporal partitioning of the Protein Data Bank (PDB) and UniProt, with evaluation sequences for newly resolved structures. The secondary setup is a type of cluster-disjoint partitioning, which puts sequences into homology clusters at 50% sequence identity using Minhash. The test sets are sampled from disjoint clusters to establish that the generative model learns underlying structural physics rather than memorizing local sequence motifs.

3.6 Target Metrics and Empirical Achievements

The structural confidence, diversity, sequence novelty and distribution coverage are used for system evaluation. A detailed evaluation is conducted to determine the performance of the model to pre-determined acceptance criteria.

3.6.1 Mean Structural Confidence (pLDDT)

The baseline acceptance criterion was a mean structural confidence of

$$\text{pLDDT} \geq 70.0. \quad (3.3)$$

The empirical results showed that unselected generations achieved a mean pLDDT of 40.63, while generation followed by post-generation candidate selection achieved a mean pLDDT of 72.89.

3.6.2 Sequence Diversity (Cluster Distance at 0.5)

The acceptance threshold for cluster distance at 50% sequence identity was

$$\text{CD}_{0.5} \geq 0.50. \quad (3.4)$$

In the standard deduplicated corpus, the model achieved

$$\text{CD}_{0.5} = 0.9920. \quad (3.5)$$

This indicates a substantially reduced level of sequence redundancy within the generated set.

3.6.3 Sequence Novelty (L1 Distance to Training Set)

A minimum normalised L_1 sequence novelty of 0.20 was established as the acceptance criterion:

$$L_1 \geq 0.20. \quad (3.6)$$

Empirical sampling produced an average L_1 distance of 0.3027, 0.4296 and 0.5425 relative to the training set.

3.6.4 Fréchet Sequence Distance

The target criterion for Fréchet sequence distance (FD-seq) was

$$\text{FD}_{\text{seq}} \leq 0.5. \quad (3.7)$$

The optimised pipeline achieved

$$\text{FD}_{\text{seq}} = 0.06, \quad (3.8)$$

which indicates alignment between the generated sequence distribution and the global distribution of natural proteins under the selected embedding-based metric.

3.7 Societal Impact and Accessibility

The ability to afford computational resources is an important factor in determining who can participate in computational biology research. State-of-the-art generative protein design has increasingly been concentrated in major industrial research centres and well-funded institutions with access to high-performance computing clusters.

This work demonstrates an alternative approach in which protein sequence generation and structural evaluation can be performed within a single-GPU environment. This provides a technical basis for making generative protein design more accessible to smaller academic groups, independent researchers, and institutions operating with limited computational resources. One of the major findings of this thesis is that high structural confidence does not necessarily require a large model architecture. Increasing the underlying protein language model encoder from 8 million to 150 million parameters, corresponding to ESM-2 t6 and t30 respectively, increased the mean structural confidence by only 1.62 points, from 71.83 to 73.45.

Increasing the model size further to approximately 420 million parameters using ProtBERT resulted in a lower structural confidence score of 68.26 under the evaluated configuration. This demonstrates that increasing parameter count alone does not guarantee improved protein generation performance.

Output diversity was primarily affected by dataset deduplication rather than computational scale. With an uncleaned and redundant corpus, the sequence repetition factor was 0.7466 and the cluster distance at 50% sequence identity was

$$\text{CD}_{0.5} = 0.3129. \quad (3.9)$$

After sequence deduplication reduced the repetition factor to 0.0873, the diversity increased to

$$\text{CD}_{0.5} = 0.9920, \quad (3.10)$$

without compromising structural foldability. These findings indicate that strategic data curation can play an important role in improving sequence diversity without relying solely on computational scaling.

3.8 Ethical Issues, Governance, and Environmental Standards

As generative biology tools become more widely available, technical frameworks must also need to anticipate the inclusion of biosecurity, governance and environmental controls. In this research, the model is used as an unconditional sequence generator, without structural and functional conditionings (no specification of targeted viral bindings, or catalytic motifs of toxins, etc.). Thus, the system will not be reengineered for the specific design of hazardous biological agents/pathogens without significant and explicit changes.

Another important governance requirement is methodological transparency. If the proxy limitations are hidden, generative models with only structural prediction metrics (e.g., ESMFold pLDDT) can be played to game the metric. The structure predictions in this study are based on a set of computational proxy metrics rather than on biological function and are not validated in a downstream wet-lab experiment. In addition, training data comes from open-access databases (PDB, UniProt), ensuring that all data provenance and academic licensing. Environmentally, it is very important to note that this multi-GPU cluster training can be avoided, which will lead to the decrease of energy usage and carbon footprints. The training requires only 18 - 36 GPU hours on a single 450W GPU, a very small fraction of the footprints of large-scale model training in industry (typically a few megawatt-hours).

3.9 Comprehensive Risk Management Framework

Due to strict constraints imposed by consumer hardware, there are operational, technical and method constraints to consider. This carry methodological dangers which should be dealt properly. The risk profile of the system consists of two types of risks: technical and operational risks; and empirical methodological risks.

3.9.1 Technical Operational Risks and Fallback Mechanisms

VRAM Saturation and Out-of-Memory Crashes

Large batch sizes in the structural prediction (ESMFold) or latent sampling process may lead to in VRAM saturation and out-of-memory (OOM) errors on a 24 GB GPU. This risk is addressed through dynamic batch slicing, sequence-length restrictions at $L = 512$, and CPU offloading if memory usage on GPU is close to 92%.

Storage I/O Bottlenecks during Latent Streaming

In latent streaming, any disk storage could be throttled if latent tensors are repeatedly referenced in memory. This is addressed with pre-allocated contiguous memory-mapped files on NVMe SSDs, which offer greater throughput and reduced access latency.

Long-Running Sampling Interruptions

Multi-hour sampling runs can cause power instability, system crashes or unsaved generation batches. In order to overcome this risk, the generated candidate batches are saved periodically using an atomic checkpointing protocol. In the implemented configuration, checkpoints are generated every 50 sampling steps, with interrupted configuration. The process will be resumed with generation without having to start from scratch.

ESMFold Local Execution Instability

Local structural prediction might not work for special sequence motifs or computational predictions of structure. The pipeline is therefore exception-driven on a per-sequence basis, in which case exceptions are isolated to some candidates failed and others were able to continue with the structural prediction. sequences in the batch.

3.9.2 Methodological Risks and Safeguards

Overfitting to Encoder Artifacts

Training the denoiser exclusively on static latent embeddings may introduce artifacts associated with the frozen encoder. To reduce this risk, small Gaussian perturbations are applied to cached embeddings during training:

$$\sigma = 0.01. \tag{3.11}$$

Single-Seed Variance Ambiguity

Evaluating model performance using a single random seed may obscure sampling variance. Structural metrics are run exactly one time and there are room for exploring multi seed variant ambiguity.

Deduplication Leakage across Data Splits

Redundancy between training and test sets can artificially improve apparent generalisation performance. To mitigate this risk, MinHash-based alignment and clustering are applied during preprocessing to ensure that training and evaluation sequences remain strictly separated.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

Our framework generates protein sequences through continuous denoising diffusion. The diffusion process takes place in the latent space of a frozen protein language model. A small learned decoder then converts the denoised latent representations into canonical amino acid sequences. Generation is non-autoregressive. All residue positions are produced simultaneously in parallel. There is no token-by-token autoregressive decoding. Structure prediction is not used during model training. It is used only after generation as an offline evaluation instrument.

The main question is how much foldable sequence quality can be achieved with a single consumer GPU. We use an NVIDIA RTX 4090 with 24 GB VRAM. We also examine the structural, distributional, and diversity trade-offs. To rigorously evaluate performance, five principal quantitative metrics are reported throughout:

- **pLDDT**: Mean per-residue confidence from a single-sequence structure predictor. The score ranges from 0 to 100. Scores above 70 indicate confident and structurally plausible predictions. Scores below 30 indicate effectively unfolded states.
- **Novelty (L1)**: Defined as $1 - \text{maximum normalised sequence similarity to any training sequence}$. Levenshtein distance is used for this calculation. It measures the degree of non-memorisation.
- **CD@0.5**: Clustering density at 50% sequence identity. It is calculated as the number of distinct clusters divided by the total number of sequences. Greedy single-linkage clustering is used with an 80% coverage constraint. Values close to 1 indicate broader sequence diversity. Lower values indicate concentration into specific sequence families.
- **CD@0.95**: Clustering density at 95% sequence identity under the same coverage constraint. It is used to detect near-duplicate collapse in generated pools.
- **FD-seq**: Fréchet distance between generated and reference encoder embeddings. The embeddings are mean-pooled and L2-normalised. Lower values indicate closer alignment with the reference distribution.

Corpus repetition is also measured using

$$\text{Rep} = 1 - Q_n(\text{unique } n\text{-mers} \div \text{all } n\text{-mers}), \quad (4.1)$$

The window length n of the substring being counted is allowed to range over $n \in \{8, 16, 32, 64\}$ and an n -mer is any contiguous set of n residues. The ratio in brackets is the ratio of the number of different n -mers in a sequence to the number of all the n -mers that the sequence can contain, so it decreases as the sequence repeats itself. Q_n represents the median of that ratio over the set of evaluated ratios, and subtracting it from one makes it a repetitiveness score, with 0 indicating that there is no internal repetition.

The framework has three horizontal execution bands, as shown in Figure 1. The top band covers corpus preparation and model training. The sequence corpus is collected, cleaned, and converted into L2-normalised latent representations. The model has three components: a frozen PLM encoder, a pre-norm Transformer denoiser, and a Transformer sequence decoder. The encoder remains frozen during training. Only the denoiser and decoder are updated.

The middle band covers candidate generation and post-generation selection. The trained pipeline generates 10,000 raw candidate sequences per run. Exact duplicates are removed. Each candidate is then evaluated using a composite selection score combining biological plausibility (30%) of the proposed framework, showing the pipeline from corpus collection and preprocessing to candidate scoring, global selection, and single-sequence structural evaluation. Selected sequences are folded using single-sequence structure prediction. Confidence thresholds are evaluated. Final statistical metrics (pLDDT, FD-seq, Novelty L1, CD@0.5, and CD@0.95) are then calculated. To isolate architectural variables from data-driven factors, the investigation was divided into three data phases:

- **The experimental phase (PDB 2000–2015):** A subset of structures that is curated, can be used only for sequences of length 128256 residues, and has been trained using 18,349 sequences. It was inexpensive to iterate and was used as the main testbed for architectural, Objectives and duration ablations, as well as selection ablations.
- **The whole PDB dataset (1971–2015):** The full structure archive covers a range of sequence lengths from 40 to 512 residues, and has 51,829 training sequences. This stage is used to verify pipeline robustness during the uncurated data test on a virtually three-year period. fold scale expansion.
- **The SwissProt 106k corpus:** A redundancy-controlled sequence corpus of 105,991 training sequences. It was deduplicated using MinHash 5-mer locality-sensitive hashing and split in a cluster-disjoint manner.

An inventory of all nine investigated framework variants and ablations is provided in Table 4.1. Each configuration was rigorously specified in Chapter 4 and evaluated in Chapter 5.

Table 4.1: Inventory of framework configurations and ablations investigated across the three experimental phases.

Variant / Configuration	Primary Variable Modified	Specified	Evaluated
Encoder Capacity (8M, 150M)	PLM depth and latent dimension D	4.2.1	5.1, 5.2
Alternative Encoder Family (420M)	Pre-training architecture family and width	4.2.1	5.1, 5.2
Cosine Noise Schedule	Replaces tangent schedule; step allocation	4.2.5	5.1, 5.2
Extended Duration (360 Epochs)	Doubles total training epoch budget	4.4.1	5.1, 5.2
Proxy Foldability Objective	Learned structure predictor hinge loss	4.4.2	5.1, 5.2, 5.6
Composition Objectives	Differentiable sequence-statistic losses	4.4.3	5.1, 5.2, 5.6
Cluster-Aware Selection	Stratified cluster retention rules	4.4.5	5.3
Unfiltered Generation	Removes selection stage entirely ($n = 2,000$)	4.4.5	5.1, 5.4
Three Training Corpora	Corpus scale and redundancy control	4.3	5.1, 5.4, 5.6

4.2 Preliminary Design or Design (Model) Specification

4.2.1 Frozen Encoder

The encoder takes sequence batches and extracts per-residue hidden representations from a specified internal layer. The BOS and EOS tokens are discarded so that latent index i corresponds directly to residue index i . Non-protein padding positions beyond the sequence length are zero-filled and excluded using boolean validity masks. Encoder parameters are fixed at load time and receive no gradient updates. Four encoder variants were evaluated:

- **Small (8M parameters):** Extracted from layer 6 (latent dimension $D = 320$).
- **Default (35M parameters):** Extracted from layer 12 (latent dimension $D = 480$), serving as the default representation model across all baseline runs.
- **Large (150M parameters):** Extracted from layer 30 (latent dimension $D = 640$).
- **Protbert Encoder(420M parameters):** Extracted from layer 31 of a distinct PLM family (latent dimension $D = 1024$).

Crucially, for each candidate-scoring operation in the post-generation selection stage, the scoring representation model was fixed to the 35M variant across all runs. This ensures that candidate likelihood scoring baselines remain directly comparable regardless of the underlying generative encoder used.

4.2.2 Latent Normalisation

Raw per-residue activations from frozen language models exhibit per-dimension variance discrepancies of more than an order of magnitude. Due to the isotropic Gaussian noise added during diffusion, unnormalised low-variance dimensions are disproportionately destroyed early in the forward process. **To ensure uniform effective noise scheduling across all latent dimensions, activations are standardized over non-padded positions:**

$$z_0^{(d)} = \frac{E(x)^{(d)} - \mu^{(d)}}{\max(\sigma^{(d)}, 0.01)} \quad (4.2)$$

In this case, $E(x)$ is the per-residue activation of the frozen encoder for input sequence x and the index (d) refers to one of the D latent dimensions. μ^d and σ^d are the means and standard deviations over real (i.e., not padded) positions across the training corpus of that dimension. The denominator of the constant is a lower bound of the divisor to avoid amplification of a near constant dimension into noise; z_0 is the normalised latent to which the diffusion process acts.

4.2.3 Denoiser

The denoiser consists of a 6-layer pre-norm Transformer encoder stack ($D_{\text{model}} = 512$, 8 attention heads, feed-forward dimension 2048, dropout 0.0). It predicts the clean latent z_0 directly instead of the added noise ϵ . Input projections map the concatenated latent and self-conditioning channels from $2D$ to 512 dimensions. Self-conditioning is applied with a probability of 0.5 during training.

Learned absolute positional encodings are incorporated. Without explicit positional encodings, the stack would be permutation-equivariant and unable to represent sequence register, secondary structure periodicity, or global chain organization under high noise levels. Timestep conditioning uses sinusoidal embeddings projected through SiLU activations into per-block gating layers. Mid-stack and penultimate activation skip connections are re-injected before the final LayerNorm and linear output projection ($512 \rightarrow D$).

4.2.4 Sequence Decoder

The sequence decoder converts full normalised latent representations into 20-class amino acid logits per position. It consists of 3 pre-norm Transformer layers ($D_{\text{decoder}} = 480$, 8 attention heads, dropout 0.1), followed by a linear projection ($480 \rightarrow 20$). Input linear projections adapt the varying encoder latent widths ($D \in \{320, 480, 640, 1024\}$) to the fixed decoder width of 480, keeping decoder capacity constant across encoder ablations. The decoder enforces 3D input tensor shapes to prevent silent execution through unbatched per-position paths.

4.2.5 Diffusion Formulation

The forward diffusion process discretizes noise addition over $T = 1000$ steps:

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (4.3)$$

In the above, z_0 denotes the clean normalised latent and z_t denotes the noised version of this latent at the diffusion time t ranging from 0 (clean) to $T = 1000$ (pure noise). The vector ϵ contains Gaussian noise, which is constructed independently for each sample, and has the same shape as the latent. The cumulative signal retention at step t , denoted $\bar{\alpha}_t \in [0, 1]$, scales the amount of the original latent remaining, while the amount of noise replacing the original latent scales by $\sqrt{1 - \bar{\alpha}_t}$.

$$\bar{\alpha}_t = \frac{1}{1 + d^2 \tan^2(\pi t/2)}, \quad d = 10 \quad (4.4)$$

The diffusion time t is represented here on the normalised time interval $[0, 1]$ where $t = 0$ represents the clean latent and $t = 1$ represents pure noise. The value of the parameter d is the steepness constant (here it is taken equal to 10) which determines the rate at which the signal is eliminated as t increases. If the total signal loss is large, a large part of it is also in small t , so that large part of the resolution of the schedule is used in the low-noise region where the denoiser is actually used to form the final sequence.

“Signal-uniform DDIM step spacing gives 225 effective sampling steps. The tangent schedule allocates 92 steps to the low-noise regime ($t < 0.1$). The cosine schedule

Table 4.2: Signal retention levels $\bar{\alpha}_t$ across timesteps for Tangent ($d = 10$) and Cosine ($s = 0.008$) schedules.

Timestep t	Tangent ($d = 10$) Signal Level	Cosine ($s = 0.008$) Signal Level
0.10	0.5339	0.9859
0.25	0.2347	0.9203
0.50	0.0995	0.7027
0.90	0.0158	0.1552

allocates only 4 of 250 requested steps. Direct prediction of z_0 under the tangent schedule places sufficient gradient mass on high-noise steps. These steps establish global structural topology. Sequence length is sampled before generation from empirical training length distributions. It serves as the attention mask throughout reverse sampling.

4.3 Data Collection:

Two collections of protein sequences of deliberately different characteristics are employed in this work. The first one comes from experimentally resolved structures and the second one is from curated sequence annotations. They do not stand for each other. They are each a different regime, and for each of them to be separated from the effect of the training data, the same pipeline has to be used with both.

Structural data (PDB). The first dataset is from the Protein Data Bank, which was exported publicly at <https://github.com/GDSC-IIIT-V/protein-classification-ml-model>. The export is split up in two files: `pdb_data_no_dups.csv` containing per-structure information like classification, experimental technique and release date, and `data_seq.csv` containing the corresponding amino acid sequences. They shared an identical `structureId` field, which was merged together using pandas’ inner method, retaining only rows that occur in both files, to create a single table with each row having a sequence and its metadata.

There were two reasons for selecting the PDB. The entries are experimentally-determined (based largely on X-ray crystallography, cryo-EM, and NMR) and refer to chains known to form a stable fold, not to the product predicted or inferred. In particular, its shorter chains are often well ordered, a property that is well suited to a study focussed on generating sequences within a bounded length range, and testing the folding. Other properties of the metadata make the collection easy to process with a generic pipeline: temporal partitioning can be done on the release date field; stratified analysis can be performed on the classification field; the same pre-processing pipeline can be targeted to different subsets without re-structuring. This is an important property of this collection for the present work; it is redundant. The PDB is not a collection of unique proteins and often a protein will be found multiple times in different entries, chains and related complexes. This redundancy does not get eliminated at source, and one of the main interests of this thesis is to

characterise the outcomes of this redundancy.

Sequence data (Swiss-Prot). The second one is UniProtKB/Swiss-Prot, the manually reviewed portion of UniProt, from the official release at https://ftp.uniprot.org/pub/databases/uniprot/current_release/knowledgebase/complete/uniprot_sprot.fasta.gz. The entire sequence was extracted from the FASTA file, including identifiers and sequences.

It is interesting to note that Swiss-Prot was selected because it is different from the PDB on the axis of interest here. It is manually curated and non-redundant at the record level and each entry corresponds to an annotated protein and not to an instance of a solved structure, and covers a much broader range of protein families than the structural archive. It has no experimental structure, so its sequences cannot be folded and have to be predicted, it provides scale and diversity that the PDB does not have.

Why both. The two collections are not necessarily used together, as it is a methodological decision. They can vary in size and the scope of families which they encompass, but most importantly by how much they repeat within themselves. The model and training procedure are the same for both, so if there is any discrepancy in generated output, it can be attributed to the data and not to the system itself. Each preprocessing method described below is then applied to each one of them, producing a set of corpora which are the ones that will be used in the ablations in Chapter 5.

4.3.1 Data Cleaning

Five sequential cleaning steps were applied to all data configurations.

1. **Protein-only filter:** Nucleic-acid and non-protein hybrid entries were excluded.
2. **Exact-duplicate removal:** Identical sequence strings were deduplicated first. Identical structure-chain pairs were then removed. Exact-sequence deduplication removed 77.8% of merged CSV rows (471,149 $\rightarrow$ 104,813). This was the single largest filter on structural data.
3. **Canonical-alphabet filter:** Sequences containing non-canonical symbols, including 'X', were removed. Retention is stricter than masking. This prevents training inconsistencies between cross-entropy and latent MSE.
4. **Length policy:** Strict length boundaries were enforced without sequence truncation. Truncation was rejected because cut proteins expose hydrophobic cores. Such proteins cannot fold properly.
5. **High-missingness column removal:** Metadata attributes with more than 75% null values were removed. No columns met this threshold.

As shown in Tables 4.3 and 4.4 and Figures 4.1, 4.2 and 4.3, the length policy accounts for the difference between the 3.89% retention in the experimental phase and the 11.00% retention in the whole PDB dataset.

Table 4.3: Stage-by-stage preprocessing attrition for the two structural data configurations.

Cleaning Stage	Experimental Phase (PDB 2000–2015)	Whole PDB Dataset
Merged CSV rows	471,149	471,149
After exact-sequence deduplication	104,813	104,813
After deposition-year window	63,938	71,911
After protein-only filter	60,177	67,628
After structure + chain deduplication	60,177	67,628
After canonical-alphabet filter	57,312	64,321
After length policy (final training set)	18,349	51,829
Held-out set (2016–2018)	4,525	11,112
Overall retained fraction	3.89%	11.00%

Table 4.4: Stage-by-stage preprocessing attrition for the SwissProt 106k corpus.

Cleaning Stage	SwissProt 106k Corpus
FASTA entries parsed	575,748
After 40–512 length prefilter	460,604
After canonical-alphabet filter	458,920
After exact-sequence deduplication	379,976
After MinHash redundancy bucketing	319,501
After holding out 5,000 whole buckets	314,501
After 128–256 length policy (final training set)	105,991
Held-out set (cluster-disjoint)	1,176
Overall retained fraction	18.41%

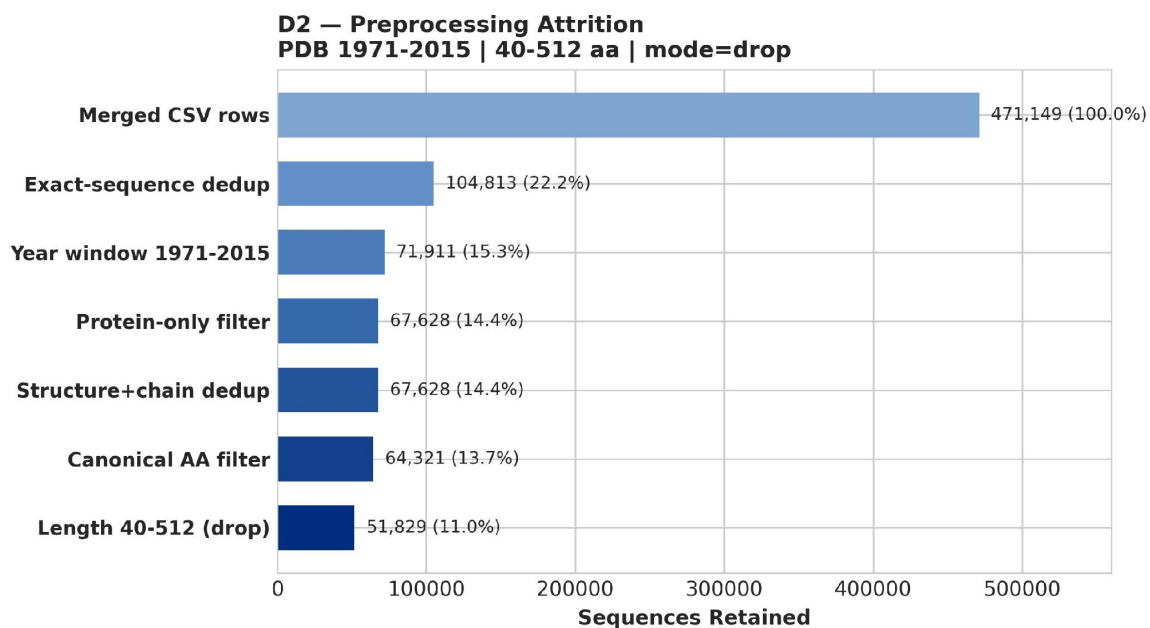


Figure 4.1: Preprocessing attrition profile for the Whole PDB Dataset (1971–2015), showing sequence retention at each cleaning stage

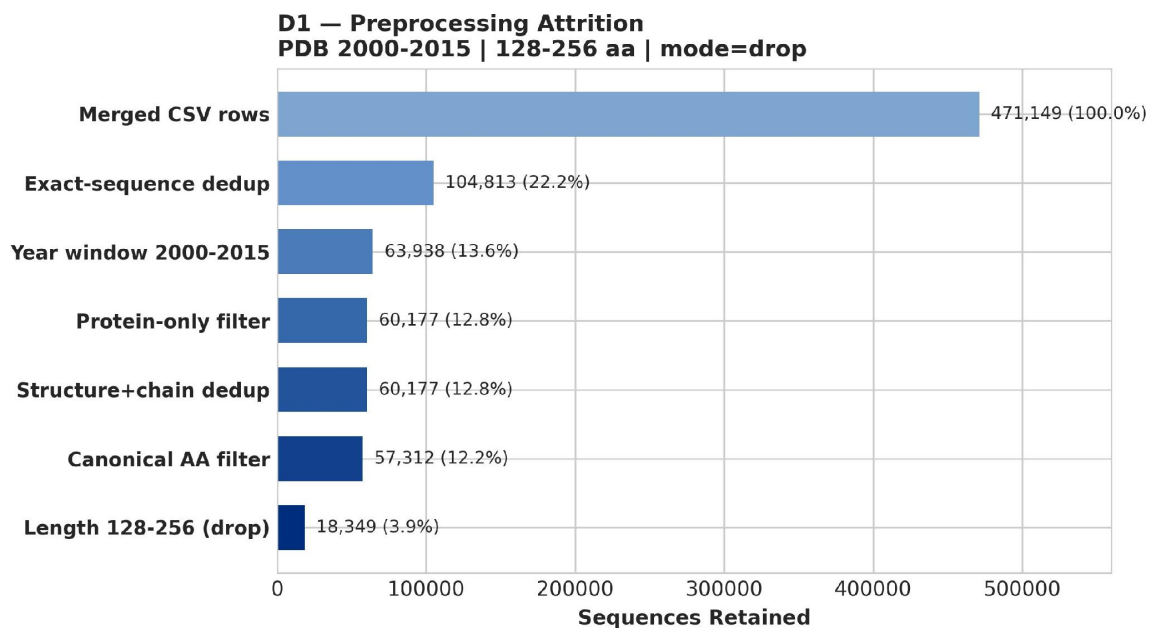


Figure 4.2: Preprocessing attrition profile for the Experimental Phase (PDB 2000–2015), showing sequence retention at each cleaning stage.

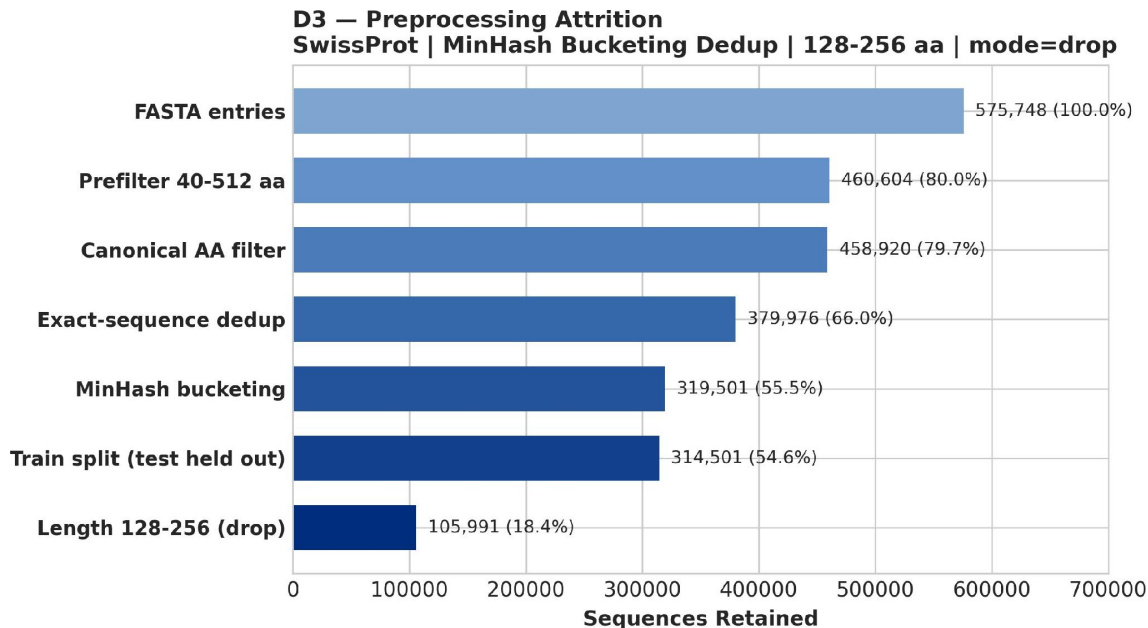


Figure 4.3: Preprocessing attrition profile for the SwissProt 106k corpus, highlighting the Min-Hash redundancy bucketing stage.

4.3.2 Data Transformation

Sequence to Latent: By passing cleaned sequences through the frozen language model encoder once, it produced latent tensors of shape $(L_{\max}, D)$ and boolean validity masks. Pre-computing latents made single-GPU training more practical.

Latent Normalisation: The standardisation was applied dimension-wise over real positions keeping standard deviations below 0.01 setting to 0.01.

Amino-Acid Targets: Sequences were mapped to integer indices 0–19 and these indices correspond to the canonical vocabulary ACDEFGHIKLMNPQRSTVWY. The padding positions use the ignore index.

4.3.3 Data Integration

For the structural sources, per-structure metadata and chain sequences were merged on structure identifiers. To separate training sets from held-out evaluation sets, temporal splits were used where training used deposition years 20002015 or 19712015, while heldout evaluation used deposition years 2016 to 2018. Numeric metadata were coerced, clipped to physical ranges, and median-imputed using training medians only. For the SwissProt corpus, sequence entries were parsed directly from FASTA, and metadata fields were set to constant default values. SwissProt split construction was enforced through cluster-disjoint partitioning by holding out 5,000 whole MinHash buckets.

4.3.4 Data Reduction

Length-Band Reduction: Removed all out-of-band sequences, discarding 68.0% of the experimental phase candidates and 66.3% of the SwissProt candidates.

Redundancy Reduction: Applied only to SwissProt using MinHash bucketing over 5-mer sets. The 379,976 deduplicated sequences were reduced to 319,501 buckets using 64 hash functions and banding on the first 4 signature values.

Candidate-Pool Selection: Post-generation filtering reduced 10,000 raw candidates to 2,000 top-ranked sequences.

4.3.5 Summary of Preprocessed Data

The characteristics of all three preprocessed datasets are summarized in Table 4.5.

Table 4.5: Summary of preprocessed dataset characteristics across all three data phases.

Dataset Property	Experimental Phase	Whole PDB Dataset	SwissProt 106k
Source	Merged PDB Metadata	Merged PDB Metadata	SwissProt FASTA
Parsed Rows / Entries	471,149	471,149	575,748
Training Sequences	18,349	51,829	105,991
Held-Out Sequences	4,525	11,112	1,176
Retained Fraction	3.89%	11.00%	18.41%
Length Band	128–256	40–512	128–256
Length Profile (p10 / p50 / p90)	139 / 192 / 242	90 / 216 / 399	141 / 192 / 243
Padded Latent Length	256	512	256
Latent Width D	480	480	480
Redundancy Reduction	None	None	MinHash 5-mer Bucketing
Partitioning Strategy	Temporal	Temporal	Cluster-Disjoint
Corpus Repetition Rep(train)	0.7466	0.7453	0.0873

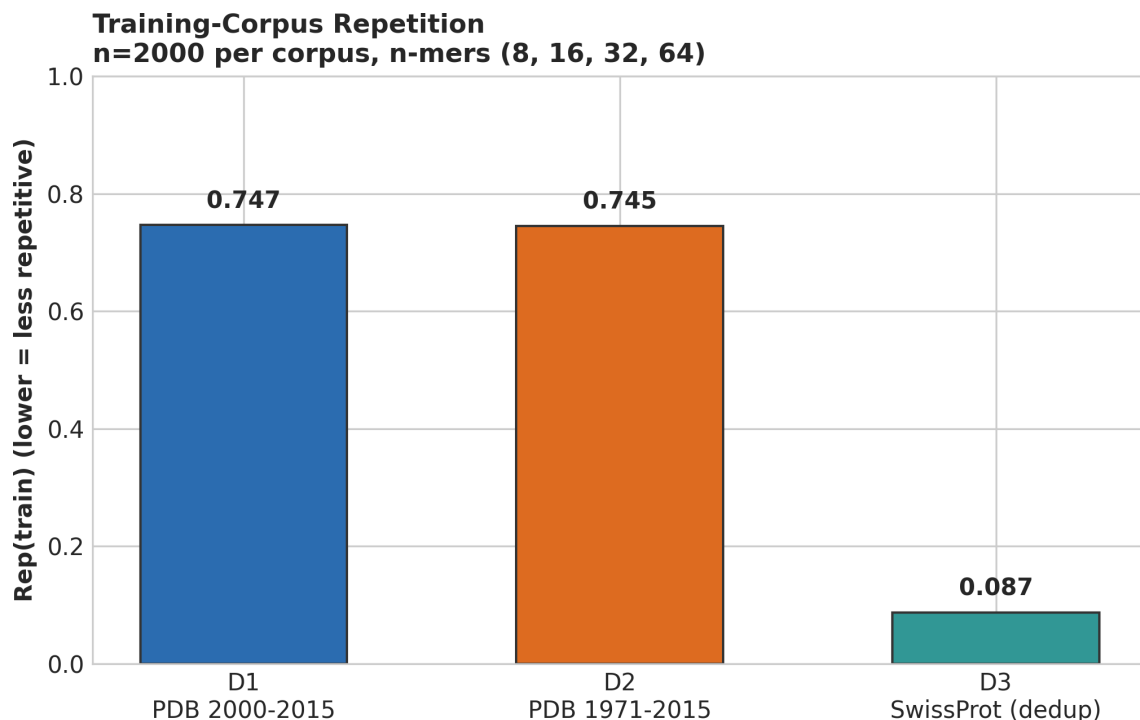


Figure 4.4: Training corpus repetition $\text{Rep}(\text{train})$ across the three datasets (evaluated on fixed samples of 2,000 sequences across $n \in \{8, 16, 32, 64\}$).

The single most consequential row in Table 4.5 is $\text{Rep}(\text{train})$, illustrated visually in Figure 4.4. The structural corpora exhibit high repetition (0.7466 and 0.7453), whereas MinHash deduplication reduces SwissProt repetition to 0.0873—an 8.5-fold reduction in internal repetition.

4.4 Implementation of Selected Design

4.4.1 Training Objective

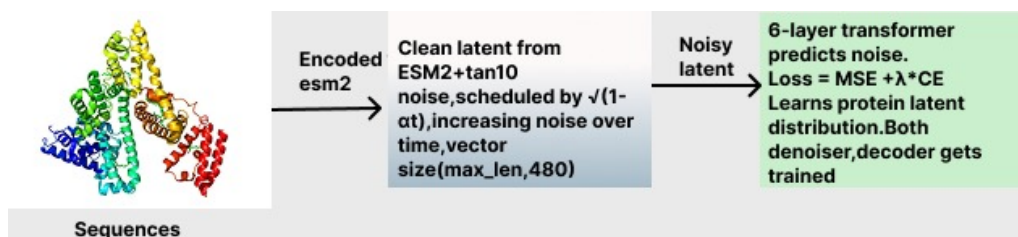


Figure 4.5: Model training process.

The denoiser and the decoder are trained together, in a common optimisation loop, and with a single objective. But, they still blame for various failures. The denoiser should recover from a noised one to a clean one; the decoder should recover from a latent to recover the residue identities. Since these are separate tasks and have separate failure modes, the intent is to only allow the terms that apply to the component to be at the component. The compound form is

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{CE}} \mathcal{L}_{\text{CE}}, \quad \lambda_{\text{CE}} = 0.2 \quad (4.5)$$

In all experiments reported in this work, λ_{CE} was fixed at the value of 0.2. The weight does not constitute a tuned hyperparameter. The two terms are forwarded to two disjoint parameter sets as illustrated below; as a result, λ_{CE} scales the decoding learning rate according to the denoiser’s learning rate and can therefore be set to one value throughout the experiment avoiding the need to adjust it when performing multi-term ablations, thus ensuring consistency.

Construction of a training step : A batch of pre-computed clean latents, their padding masks and the residue targets y are computed for each step. A timestep index is sampled uniformly and independently across each sequence, $t \sim \mathcal{U}\{0, \dots, T-1\}$; as a result, each batch of samples includes examples from all noise levels and not just a single noise level. The noised latent variables are generated in the forward process.

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (4.6)$$

As in (4.3), z_0 is the clean latent, and z_t denotes the latent after t noise steps have been added to it, with the same notation for ϵ and $\bar{\alpha}_t$ as in (4.3). Each training example is given a step index t , which is sampled uniformly at random, thus training in one batch spans a variety of noise levels.

The denoiser first is run once without self-conditioning input, and its output (clamped and detached) is fed in as a second control input of the second (gradient carrying) pass. The detachment is important: The self-conditioning is intended to provide an additional gradient path for the denoiser to look at, not duplicate the previous forward pass gradient path. The other half of the steps is not self-conditioned, so that the model does not become reliant on it.

The reconstruction term:

The denoiser directly estimates the clean latent, and is evaluated using masked mean square error at unpadded positions:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{D \sum_i m_i} \sum_i m_i \|\hat{z}_0^{(i)} - z_0^{(i)}\|^2 \quad (4.7)$$

The denoiser’s prediction of the clean latent is $\hat{z}_0$, while z_0 is the true clean latent and the superscript (i) denotes the position of each residue. In the binary mask m_i values are 1 for a real residue position and 0 for padding positions, which means padding will contribute nothing to the loss. The latent width is denoted by D and the loss per scalar element is normalised by dividing by $D \times \sum m_i$ to make sure that long and short sequences contribute equally to the loss per residue.

The normalisation is done per element, that is, the sum is divided by the number of valid positions times the latent dimension D ; the loss is an average loss over scalars, not sequences. Otherwise, longer sequences would add a higher proportion of gradient to the system and batches would be weighted by the ease of sequence rather

than length. No reweighting is performed per time step, the loss mass is unweighted and the distribution of loss mass over noise levels is controlled by the noise schedule as described in the previous section.

This quantity is not too large to interpret as the latents are normalised. The MSE is close to 0.91, which is the MSE of a degenerate model that emits no latents, for the normalised target. Trained denoisers fit in the range 0.63 – 0.69 – which is about a $\frac{1}{4}$ to a $\frac{1}{3}$ of the available signal. This is used later as the basis for the bottleneck analysis.

The decoding term:

The decoder is jointly trained with two inputs:

$$\mathcal{L}_{\text{CE}} = 0.5 \text{CE}(g_\phi(z_0), y) + 0.5 \text{CE}_w(g_\phi(\text{sg}[\hat{z}_0]), y) \quad (4.8)$$

The ground truth amino acid label for each position is denoted by y and the sequence decoder with parameters ϕ is denoted by g_ϕ . The first branch supplies the decoder with the clean latent z_0 , while the second branch supplies the decoder with the denoiser’s estimate $\hat{z}_0$, which is what the decoder actually receives at the generation time. The stop-gradient $\text{sg}[\cdot]$ is used to ensure that the branch trains the decoder alone, not the denoiser and CE signal-weighted cross-entropy in (4.9). Equal parts of the two branches are combined.

Both the branches are used for different purposes. The first one provides a stationary, clean learning problem: If one is given the correct latent variable, what are the residues? This is the decoder’s notion of “correctness” and that’s how it attains almost perfect accuracy on ground-truth latents. The second involves perturbing the decoder input at generation time such that it is the imperfect estimate of the denoiser, rather than a clean latent. When trained on the first alone, the decoder will become accurate on the second type of inputs, and then when trained on the second alone, the target will drift with the improvement of the denoiser, and there will be no stable signal early in training when $\hat{z}_0$ is close to the noise. Both properties are retained in the equal mixture.

Signal weighting. The second one is a non-ordinary cross entropy. The loss for each position is scaled by the signal retention of the timestep which created its latent:

$$\text{CE}_w = \frac{\sum_{b,i} w_b v_{b,i} \text{CE}_{b,i}}{\sum_{b,i} w_b v_{b,i}}, \quad w_b = \sqrt{\bar{\alpha}_{t_b}} \quad (4.9)$$

b is the index of the sequence in the batch, and i is the index of the residue position, so $\text{CE}_{b,i}$ is the cross-entropy per position of this residue. $v_{b,i}$ is equal to 1 if it is in a valid position (not a padded position) and 0 otherwise. The signal retained at the timestep t_b to noise that particular sequence is the weight $w_b = \sqrt{\bar{\alpha}_{t_b}}$ which approaches 1 for light noise and 0 for heavy noise, while focusing the term where the identity of the residue is truly recoverable.

The valid-position indicator should be denoted by the word where. The weight is close to one for light noised latents and close to zero for heavy noised latents. The reason for this is the sampling trajectory: Only at the end of a generation, when a latent has already been through the low-noise regime, the decoder is called. The identity of the residue predicted from a heavily noised latent is in general not recoverable, and the decoder will waste capacity on a sub-problem which is in principle unsolvable, and will harm the accuracy of the rest of the model if it does. The weighting focuses this term at the point where the decoder’s output has a meaningful impact.

Gradient routing : The reason why the two components stay separate is the stop-gradient in the second branch. The denoiser is supervised only with $\mathcal{L}_{\text{MSE}}$ since there is no decoder gradient reaching the denoiser due to the fact that $\hat{z}_0$ is decrypted ahead of entering the decoder. However, this means that decoder branches are unable to affect what is learned by the denoiser. This is on purpose: otherwise the denoiser might use the easy-to-decode latents and minimize the combined loss, resulting in a decoding shortcut for the diffusion objective. The routing may be succinctly expressed as

$$\nabla_{\theta}\mathcal{L} = \nabla_{\theta}\mathcal{L}_{\text{MSE}}, \quad \nabla_{\phi}\mathcal{L} = \lambda_{\text{CE}}\nabla_{\phi}\mathcal{L}_{\text{CE}} \quad (4.10)$$

Each set of parameters are denoted by the variable θ for denoiser parameters, ϕ for decoder parameters, and $\mathcal{L}$ denotes a gradient of the total loss with respect to each set. The denoiser is only updated by the reconstruction and the decoder is only updated by the weighted cross-entropy, this is ensured by the stop-gradient in (4.8). Both are updated in the same optimiser step, hence no stale copy of the other is used for training.

where θ is the denoiser parameters and ϕ is the decoder parameters. The separation is important in that it enables auxiliary objectives to be targeted specifically at the detached decoder branch – the composition objective(s) are only affecting the readout while the proxy-foldability objective is not detached to enable the gradient to reach the denoiser.

Two auxiliary objectives were also investigated:

4.4.2 Proxy Foldability Objective

As described above, reconstruction and cross-entropy terms act as supervision for the model, with respect to a single target latent and associated residue identities. There is no indication in the sequence that the decoder would provide is structurally plausible. The natural supervisory signal of that property, pLDDT, can’t be used as a loss directly since it comes from a structure predictor that is not differentiable with respect to the latent. The proxy foldability objective aims to tackle this by using a small learned model that approximates the structure predictor, which is constructed in a way to be differentiable.

A confidence predictor f_ψ is fitted once at the beginning of the diffusion training and is then kept frozen. Maps a masked latent sequence to a single scalar on the normalised pLDDT scale:

$$f_\psi : \mathbb{R}^{L \times d} \times \{0, 1\}^L \rightarrow [0, 1]$$

The predictor is a 3-layer transformer with 256 hidden width and mean pooling over the valid positions which contains 2.62M parameters. It is trained over 2,450 pairs of latent-pLDDT for 40 epochs, under mean squared error. The set of labels is deliberately heterogeneous. Real sequences lie in a small confidence interval around 80 and a predictor determined on that interval approaches a constant function with a gradient of 0. The low end of the range are composition-matched shuffled sequences and random sequences; the informative sequences are the shuffled ones: They are identical in amino acid composition, only the order of the residues is changed, forcing the predictor to read the structural arrangement rather than the number of amino acid types. At training time, the predictor will be applied to $\hat{z}_0$ (the clean latent plus denoiser error) instead of a clean latent, hence, each latent is perturbed with Gaussian noise with randomly sampled scale $\sigma \sim \mathcal{U}(0, 0.4)$ during the fitting procedure.

The fitted predictor is attached to the diffusion objective as a one-sided hinge:

$$\mathcal{L}_{\text{proxy}} = \mathbb{E} \left[\max \left(0, \frac{\tau}{100} - f_\psi(\hat{z}_0, m) \right) \right], \quad \tau = 93.0 \quad (4.11)$$

There are three properties of this formulation which are deliberate. First, the one-sided hinge means that the term fits no gradient when the predictor passes the hinge, and it will never push the model beyond the target into a high-confidence region. Second, the term is computed on the denoiser’s prediction $\hat{z}_0$ and the gradient is not separated. The intention behind the objective is that the signal will be sent to the denoiser and that the signal’s output will be directed towards the portion of the latent space that the predictor has linked to foldable proteins. While this term can say go where proteins fold well, reconstruction can only say go to this particular target. Third, the term is stored in a latent state for the first 30 epochs, so that the decoder can learn to read the latent before any forces of folding are applied.

The full objective function is the reconstruction and cross-entropy functions described above in addition to this term:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{CE}} \mathcal{L}_{\text{CE}} + \lambda_p \mathcal{L}_{\text{proxy}}, \quad \lambda_{\text{CE}} = 0.2, \quad \lambda_p = 0.05 \quad (4.12)$$

The weight is 0.05, making its contribution about five percent of total loss. The known risk is clearly expressed: f_ψ is a proxy, and a model that is optimized for a proxy may learn to satisfy that proxy instead of that property. Adversarial latents which score well by f_ψ but don’t fold well are possible in principle, hence the weight is small, and the fact that the objective is not judged by the predictor’s readings of the output but by folding the output.

4.4.3 Bio Composition Objectives

The proxy objective is addressed with a learned model of foldability. The composition goals instead focus on three statistics of the emitted sequence that can be calculated in closed form, each one of which corresponds to one of the criteria for the post-hoc biological plausibility score at selection time. All three are connected to the decoder’s logits on $\hat{z}_0$ and, similar to the cross-entropy term, only the decoder branch is trained; the denoiser’s goal is to reconstruct the masked loss.

Discrete evaluation: A composition statistic based on a probability distribution is not the statistic of the sequence that the decoder would output. A position (x) in between the two residues will score the same as a confident prediction, and so a model can reduce such a loss by becoming less certain rather than more correct. The objectives are consequently scored a straight-through one-hot estimate. It is the quantity $p = \text{softmax}(\ell)$ for the per-position distribution of the decoder that is being scored.

$$\tilde{p} = \text{onehot}(\arg \max p) + p - \text{sg}(p) \quad (4.13)$$

Here $\text{sg}(\cdot)$ denotes the stop-gradient operator. The forward pass takes the discrete sequence that the decoder would emit, and gradients go through the softmax. This means that the training-time statistics are numerically the same as the post-hoc scores, which was verified to a maximum deviation of 8.1×10^{-8} .

Empirical reference levels: The terms are anchored to a level of the training corpus, not on a theoretical maximum. The reason is that the theoretical optima are not desired targets. A k -mer divergence of zero would mean using k -mers more regularly than real proteins, which is repetition but not plausibility, and a normalised entropy of one would mean flat residue composition, while real proteins are near 0.96 with strongly unequal residue frequencies. They are determined to be at the quantiles of the real distribution and are selected to ensure that a meaningful proportion of real sequences is in violation and that the hinges do not saturate early on in the training process but maintain a gradient.

k -mer divergence: The distribution of three-mers of the emitted sequence is denoted by q_3 and is estimated by contracting adjacent position distributions over the total of 8,000 possible 3-mers, while the distribution of three-mers in the reference sequence is denoted by p_3 and is estimated from 10,000 training sequences. The term penalises divergence from corpus usage:

$$\mathcal{L}_{\text{kmer}} = \max(0, D_{\text{KL}}(q_3 \| p_3) - \kappa), \quad \kappa = 3.0678 \quad (4.14)$$

The divergence is taken in this direction to have the term penalise those 3-mers that are not used in the corpus; the other direction would require a single sequence of a few hundred residues that would cover all 8,000 bins, which is unsatisfiable. The threshold is taken as tenth percentile of the divergences in real sequences, which means that 90% of real training sequences violate the threshold.

Polar fraction: Suppose that the fraction of emitted residues in the polar set $\{D, E, R, K, H, N, S, Q, T\}$ is denoted by f_{pol} . The term is a two-sided hinge on an

interval:

$$\mathcal{L}_{\text{polar}} = \max(0, \pi_{\text{lo}} - f_{\text{pol}}) + \max(0, f_{\text{pol}} - \pi_{\text{hi}}), \quad [\pi_{\text{lo}}, \pi_{\text{hi}}] = [0.4417, 0.5044] \quad (4.15)$$

The band is the interquartile range (IQR) of the polar fraction measured over the actual sequences, thus 50% of actual training sequences fall outside of the band. Here, a two-sided form is used because in both cases the presence of polar content is not desired: too little suggests an implausibly hydrophobic sequence while too much suggests an unstructured sequence.

Composition entropy: Suppose that the marginal residue distribution of the emitted sequence is q_{aa} and the Shannon entropy of the sequence is H . The penalization for compositional collapse is:

$$\mathcal{L}_{\text{ent}} = \max(0, \eta - H(q_{\text{aa}})), \quad \eta = 0.9602 \quad (4.16)$$

The threshold is the 90th percentile of real-sequence entropy, leaving 90% of real sequences to be in violation. The term is one sided and cannot “force” composition beyond real data towards the uniform distribution.

Combined objective: These 3 terms are all given equal weight in the reconstruction and cross-entropy terms:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{CE}} \mathcal{L}_{\text{CE}} + \lambda_k \mathcal{L}_{\text{kmer}} + \lambda_{\pi} \mathcal{L}_{\text{polar}} + \lambda_{\eta} \mathcal{L}_{\text{ent}} \quad (4.17)$$

with $\lambda_k = \lambda_{\pi} = \lambda_{\eta} = 0.10$. They are not like the proxy objective, they are active from the first epoch. A previous configuration used 20 epochs of warmup but by 2-5 epochs the decoder was able to reach the mean of all the statistics purely by means of reconstruction learning; at the point of the terms being activated the decoder had nothing to correct and was reading the same three 0’s for the rest of the training. Applying them from the outset, against strict quantile thresholds, keeps them alive throughout.

Optimisation:

The two components share the same AdamW optimiser (learning rate of 10^{-4} , weight decay of 10^{-5}) and are updated the same time, which allows both parameters to update simultaneously, even though they are separated. They share the same AdamW optimiser and are updated at the same time, which makes both parameters advance together, and the denoiser’s decoder is not learning from a stale denoiser. Gradients are clipped together in the same way: to a global standard of 2.0. The learning rate is linearly warmup for 500 steps to cosine annealing with warm restarts every 15 epochs. The weights of the denoiser are averaged exponentially throughout with 0.9999 as a decay factor and that is the weight for use with all generations as the instantaneous weights at the end of a cosine cycle may not necessarily be the best point along the trajectory. The decoder is used at its last weights, trained on an objective that is supposed to be stationary and will not have the cycle structured variations.

4.4.4 Generation

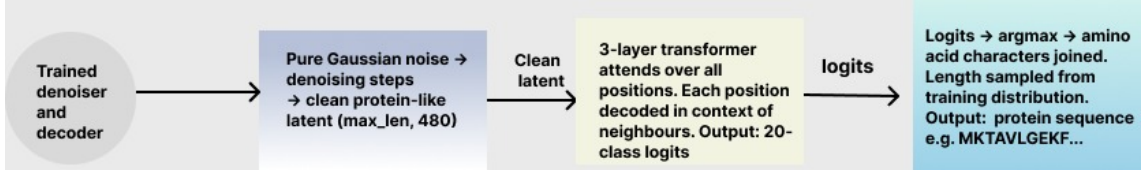


Figure 4.6: Sequence generation after model training.

Candidate generation produced 10,000 sequences per run. Batches contained 1,024 sequences. Sampling used 250 DDIM steps with seed 42. Sequence lengths were sampled individually from empirical training length distributions. The joint frequency of alanine, cysteine, aspartate, and glutamate was 20.4%–22.0% across generated pools. The training set range was 20.9%–22.1%. Following the exact same procedure, 2000 sequences were produced for unselected result ablations.

4.4.5 Selection Stage

Score definitions:

1. BioScore

The BioScore integrates three complementary metrics, with each reflecting a distinct aspect. element of an authentic protein sequence. The ultimate score is an average that is weighted:

$$\text{bio_score} = 0.34 \times \text{kmer_score} + 0.33 \times \text{polar_frac_score} + 0.33 \times \text{aa_entropy_score} \quad (4.18)$$

The weights (34%, 33%, 33%) indicate the significance of local sequence patterns, balance of residue polarity and overall diversity of amino acids.

Kmer_score:

Calculation: A dataset of actual protein sequences (sourced from the same origin as the training data, such as previously resolved structures, is utilized to create a reference frequency table of every 3-mer (tripeptide) frequency. For a novel sequence, the occurrence of every 3-mer within it is computed. For each 3-mer in the sequence, the score increases by:

$$p_{\text{seq}} \times \log \left(\frac{p_{\text{seq}}}{p_{\text{train}}} \right) \quad (4.19)$$

where p_{seq} represents the relative occurrence of that 3-mer in the sequence, and p_{train} indicates its occurrence in the training dataset (adjusted with a minor epsilon to prevent division by zero). The total for all 3-mers is subsequently converted into a score ranging from 0 to 1 by:

$$\text{score} = \max \left(0, 1 - \frac{|\text{sum}|}{5} \right) \quad (4.20)$$

Understanding: The sequence is similar to the occurrence of natural proteins. If a sequence contains at least one peptide at frequencies strikingly close to natural proteins, the resulting mathematical discrepancy As the number gets smaller, then kmer_score approaches 1.0. In contrast, misbehaving or unusual 3-mer arrangements that are highly repetitive or structurally anomalous residue combinations, induce significant difference from one another, so that the score keeps dropping. The sequence of this short 3-mer window is very effective in that it successfully works at a localized, shortrange level on residue interactions which are fundamental to secondary structure formation.

Polar_fraction_score:

Calculation: This is a measure of the balance of hydrophobic and hydrophilic properties of a sequence. Amino acids are specifically classified as polar (D, E, R, K, H, N, Q, S, T) or nonpolar (A, C, F, I, L, M, P, T, V, W, Y). This function calculates precise percentage of polar residues through the pipeline. If this ratio is within the 0.35 to 0.65 range then a score of 1.0 is given. If the percentage falls below or above this threshold, there is a linear penalty in score

$$\text{score} = \max(0, 1 - 4 \times |\text{frac} - 0.50|) \quad (4.21)$$

In nature, water-soluble proteins require a delicate balance in the spatial arrangement of surface-exposed, buried, non-polar residues and polar residues. If there is too many of a given word, if it has a high concentration of hydrophobic residues, it will certainly aggregate, and if it is highly polar, it will certainly disperse. A stable, hydrophobic core structural support is not possible with sequence. Because typical For soluble globular proteins, the polar distribution is about 40-60%, which is expected, Very high and very low values are not given any extra weight for benchmark. This enables the framework to ensure important. Using a relatively simple 3D structure, this method requires limited computation power, but can predict the biophysical boundaries early on. predictions.

aa_entropy_score Calculation:

Assuming a sequence contains the specific counts C1...C20 for each of the 20 standard amino acids, its Shannon entropy is calculated using the following formula:

$$H = - \sum_i \left(\frac{c_i}{L} \right) \times \log \left(\frac{c_i}{L} \right) \quad (4.22)$$

In this equation, L represents the total length of the sequence. Given that the absolute maximum entropy achievable for 20 distinct symbols is $\log(20) \approx 3.0$, the final scaled metric is defined as:

$$\text{score} = \min \left(1, \frac{H}{\log(20)} \right) \quad (4.23)$$

Protein sequences in nature do not rely on just a few types of amino acids, because this would result in the reduction of entropy. On the other hand, they do not divide the composition evenly among all twenty types, since this would be the case for maximal entropy. The pattern which is usually seen is the diversity level in between these two extremes. The low-entropy sequences, such as a sequence of alanines, cannot form a distinct structure, while the maximum-randomness sequence is usually

chemically unorganized.

2. ESM-2 Pseudo Log-Likelihood (PLL):

We decided to use ESM-2 pseudo-log-likelihood (PLL) as selection metric, following prior work that showed the effectiveness of PLL and pseudo-perplexity for protein fitness estimation and foldability, directly related to pLDDT.[47], [48]. A well-balanced diversity that is both structurally stable and functional. Sequences with low entropy (e.g., a poly-alanine chain) generally are not able to fold into a random, largely non-functional 3D structure, while sequences with maximum randomness. As a result, this is a measure that assesses and rewards sequences that show a realistic spectrum of residue varieties, which resemble the diversity profiles. ESM-2 (Evolutionary Scale Modeling) is a transformer-backed protein language that documents in natural proteomes. model trained on millions of natural sequence. In sequences, the model is able to self-learn to figure out what the missing amino acid is, completely from the rest of the amino acid sequence. On the surrounding context, practicing the basic 'grammar' of biology. The framework takes advantage of this particular predictive capacity to assess the degree of proximity of a Newly created sequence follows natural rules. Calculation Steps Each generated sequence goes through the following steps to be evaluated:

1. Length Truncation : Some of the sequence is cut off and cannot be displayed because it exceeds the maximum number of tokens.

2. Token Initialization : The start-of-sequence and end-of-sequence tokens) are explicitly added to the beginning and end of the data stream. All the necessary structural markers (namely start-of-sequence and end-of-sequence tokens) are explicitly inserted at the start and end of the data stream. The sequence is followed by end-of-sequence tokens (as needed for ESM-2).

3. Subsampling and Masking : A set of amino acid positions is selected by a certain stride format. For example if you use a stride of 5, then 20% of the entire sequence positions will be applied for testing. For each of the positions chosen in this step, Amino acid residue at that particular position is replaced by a dedicated [MASK] token. This masked sequence is passed to esm2 The system takes the output logits on the masked position and isolate the exact log probability (via log-softmax) the model gives back to the true, original amino acid. - These extracted log probabilities are then systematically added up across all sampled positions.

4. Likelihood Averaging : The total sum is divided by the number of masks. positions, giving an average log-likelihood score per tested token.

5. Batch Normalization : Across the whole batch is performed as a final min-max normalization step is done across the entire Produces a scaled plddt_proxy metric ranging from 0 to 1 for a batch of generated sequences. These values range from to 1 (with higher being better).

Interpretation: What the PLL does is basically compute the amount of “surprise” that ESM-2 demonstrates in the face of the corresponding amino acids at each po-

sition sampled. The log probability of a structural pattern that is seen frequently in nature will always be high. This (a low surprise) is eventually reflected in a high PLL score. On the other hand, if a sequence brings highly uncommon configurations (like an unanticipated), it can make the configuration more difficult to handle. unusual pairing of peptides or an uncharacteristic residue in a particular structural context. The overall PLL will be lowered by model since it will give it a low probability. The ESM-2 contains proven ability to implicitly capture underlying structural and functional changes, It shows strong correlation with overall sequence foldability, when its internal PLL is considered in the context of biological constraints, genuine “proteinlikeness”. In this context, the scaled PLL is simply a min-max scaled PLL. To calculate a model that is a reliable proxy for pLDDT the precise measure of confidence that a complete structure prediction model would yield. Generate if it was to fold the sequence. This enables sequences to have high PLL markers in them which has been termed as highly viable candidates, are likely to be high-confidence 3D structures. Effective pre-computing intensive downstream folding. The rationale behind the use of both metrics together. which has been termed as highly viable candidates, are likely to be high-confidence 3D structures. Effective pre-computing intensive downstream folding. The rationale behind the use of both metrics together.

Why Both Metrics Are Used Together

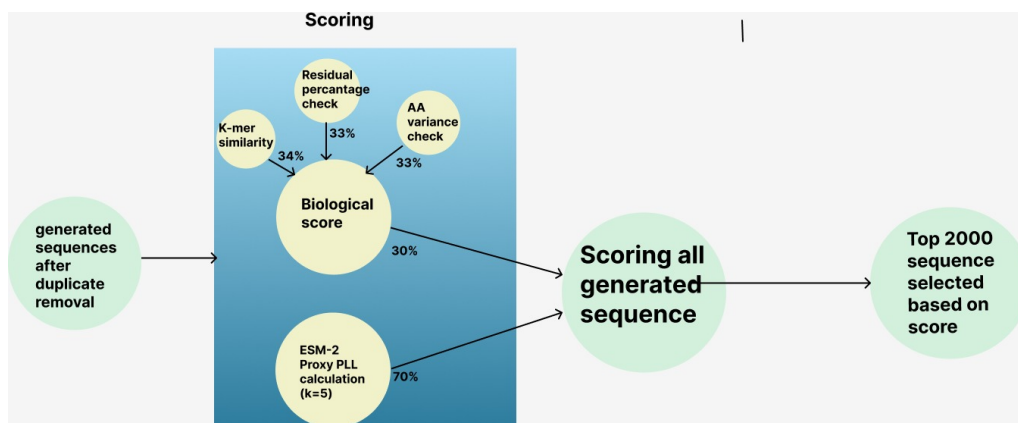


Figure 4.7: Scoring process.

- **BioScore:** This metric is very efficient and requires no GPU. Its primary role is to ensure basic biophysical restrictions, such as localised residue, are not violated. hydrophobic-hydrophilic balance, diversity, and distributions. As a result, it is very good at filtering out sequences that have no protein-like characteristics as 32 very repetitive strings or as completely hydrophobic strings.
- **ESM-2 PLL:** This language model metric requires a much larger amount of data. it is able to perform well on the understanding of complex, context-dependent patterns, Simple statistical counts do not account for. It is a kind of 'big picture' review. In the protein language, “fluency” means the absence of any restrictions on language. Integrated Strategy These different methods are integrated in the framework, with a weighted final equation:

$$\text{combined_score} = 0.3 \times \text{BioScore} + 0.7 \times \text{PLL} \quad (4.24)$$

The processing pipeline combines them in this way, taking advantage of the very fast speed. The BioScore’s purpose is to eliminate obvious structural abnormalities first, so that only the remaining healthy sequences are kept. Both the score values are calculated for each candidate and then aggregated into the weighted sum formula given above in such a way that the BioScore filters out the biophysical outliers and the linguistic model makes decisions about how to sort out the remaining ones.

Just for the clustering pipelines, HDBSCAN was used, with fallback of K-means clustering. This clustering was done after scoring the 10000 sequences. For HDBSCAN, we used eom mode, for the fallback K-means clustering, $K = 20$. For HDBSCAN, minimum cluster size was set to 30, if generated sequences failed to produce at least 20 clusters of ≤ 30 sequence, it was designed to fall back to K-means. Before clustering, using ESM2 mean pooled embeddings of 480D was reduced to 50 by PCA, then 50 to 15 via UMAP. This dimension reduction was done to strip noise and preserve local neighbourhood structures and pull the subclusters apart. In 480 dimension, every point maintains uniform distance among each other. For score based clustering, clusters with high score were prioritized and higher portions were selected from them, while lower score clusters got less sequences selected and for uniform clustering, we selected 20% of sequences from each cluster, then took the top 2000 from the selected ones.

4.4.6 Evaluation Implementation

Single-sequence structure prediction tools were used for each sequence. Per-residue pLDDT confidence scores were obtained. Sequences with failed structure predictions were assigned pLDDT 0.0. FD-seq was calculated as Fréchet distance over mask-aware encoder embeddings. The embeddings were mean-pooled and L2-normalised. CD@0.5 and CD@0.95 were computed using greedy single-linkage clustering. The identity thresholds were 50% and 95%. An 80% length-coverage constraint was used. Novelty (L1) was calculated as $1 - \text{maximum normalised Levenshtein similarity}$. Matched unfiltered evaluation arms used 2,000 candidates. They were run for all three phases.

4.4.7 Code and Data Availability

All code for preprocessing, training, generation, selection, and evaluation is publicly available at:

<https://github.com/Shahriar-Khan-Shirso/Protein-Sequence-Generation-and-Selection-for-Balanced-Novelty-and-Structural-Confidence>.

The repository includes the configuration corresponding to every ablation reported in Chapter 5, so that each row of Tables 5.1 to 5.4 can be traced to the exact settings that produced it. Model checkpoints are hosted separately on HuggingFace; links are given in the repository README.

Chapter 5

Result Analysis

5.1 Performance Evaluation

Performance was evaluated on the top-selected 2,000 sequences in all configurations. Unfiltered or unselected cases are stated explicitly. Diversity, novelty, and FD-seq were evaluated on folded sequences with pLDDT > 30 (n). Unfolded sequences were treated as noise rather than protein structure. All reported FD-seq metrics use the L2-normalised scale against training reference sets.

5.1.1 Experimental Phase (PDB 2000–2015)

Ablation experiments on the curated structural subset covered encoder capacity, encoder family, training duration, noise schedule, proxy objectives, composition objectives, and selection criteria. The baseline configuration used a 35M encoder, tangent schedule, 180 epochs, and global selection. It achieved a mean pLDDT of 72.89, an FD-seq of 0.2661, a Novelty (L1) of 0.3027, a CD@0.5 of 0.3129, and a CD@0.95 of 1.0000. Real training sequences had a mean pLDDT of 78.87 under the same structure predictor. Threshold distributions reveal structural properties that mean pLDDT alone can obscure.

Baseline (35M, 180 ep): 1,580 sequences were above pLDDT 70 (79%). 493 were above 80 (24.7%). Mean pLDDT being 72.89, with 30.27% novelty and 0.31 CD 0.5.

Cosine Schedule: 1,945 sequences were above 30. 1,559 were above 40. 1,288 were above 50. Only 80 sequences were above 70 (4.0%). No sequences were above 80. The mean pLDDT was 54.02. Cosine scheduling raises the quality floor. It eliminates the quality ceiling.

Proxy Foldability Objective: It yields 1,996 sequences above 30. The mean pLDDT is 74.56 (+1.67 over baseline). CD@0.5 drops to 0.2415. FD-seq increases to 0.3269.

Bio Composition Objectives : It yields 1,368 sequences above 30 and 76 above 80. The mean pLDDT is 40.69. This is statistically indistinguishable from the untouched unfiltered control, which has 1,382 sequences above 30, 97 above 80, and a mean pLDDT of 40.63.

5.1.2 Whole PDB Dataset (1971–2015)

The complete structural archive covered 40–512 residues and 51,829 training sequences. It was evaluated for pipeline robustness at scale over 180 epochs. Selected 180-epoch outputs achieved a mean pLDDT of 71.49. The median was 75.28. The standard deviation was 12.80. The range was 27.49–91.18. FD-seq was 0.1442. Novelty (L1) was 0.4296. CD@0.5 was 0.6578. CD@0.95 was 1.0000. Threshold distributions confirmed 1,999 sequences above 30 (100.0%). 1,803 were above 50 (90.1%). 1,311 were above 70 (65.5%). 584 were above 80 (29.2%). The matched unfiltered run achieved a mean pLDDT of 48.06. The median was 42.54. The standard deviation was 17.25. FD-seq was 0.0413. Novelty (L1) was 0.6325. CD@0.5 was 0.9287. 1,810 sequences were above 30, with 152 above 80. Spearman rank correlation between the composite score and true pLDDT was $r_s = 0.624$. Pearson correlation was $r = 0.590$. Generated repetition was $\text{Rep}(\text{gen}) = 0.072$. Corpus repetition was $\text{Rep}(\text{train}) = 0.7453$.

5.1.3 SwissProt 106k Corpus

The redundancy-controlled SwissProt corpus contained 105,991 training sequences with lengths of 128–256 residues. It was used to evaluate generation on deduplicated data. The real dataset reference pLDDT was 80.7.

Extending training from 135 to 180 epochs improved selected mean pLDDT from 66.67 to 68.64. At 180 epochs, 1,039 sequences were above 70 and 317 were above 80. At 135 epochs, the counts were 900 and 205. As we very initially set out pLDDT target to 70, we increased 45 epoch and regenerated sequences again, we had his headroom because previous novelty and diversity both were quite high so lessening them a slight bit for the pLDDT gain seems fair. And with epoch 225, we got pLDDT mean 71.73. pLDDT over 70 and 80 for sequences count are 1189 and 455 respectively.

Diversity remained near-perfect. CD@0.5 shifted from 1.0000 to 0.9975 then finally 0.9920 at epoch 225. Novelty (L1) shifted from 0.5906 to 0.5633 then finally at 0.5425. FD-seq shifted from 0.0521 to 0.0600 then at 225 epoch, it went to 0.065. Test-set FD-seq shifted from 0.0861 to 0.1097. The selected 180-epoch run had zero folding failures. All 2,000 sequences were above pLDDT 30, same with 225 run-epoch.

The unfiltered run with 180 epoch’s checkpoint achieved a mean pLDDT of 50.35. The median was 46.52. The standard deviation was 14.87. The range was 22.26–90.04. 1,952 sequences were above 30, 270 were above 70, and 73 were above 80. FD-seq was 0.0361. Novelty (L1) was 0.6633. CD@0.5 was 1.0000.

For the unselected generation for epoch 225, pLDDT rised from 50.35 to 52.17, the median was 49.20. The pLDDT range being 23.84–91.57. 1,950 sequences were above 30, 329 were above 70, and 100 were above 80. FD-seq was 0.029. While novelty dropped by 1%, the 225 epoch sequences are 65.3% novel, the cluster diversity metrics remained at 1.00.

5.1.4 Unselected Generation

Unfiltered generation across all three phases ($n = 2,000$ raw candidates folded directly) measures the contribution of post-generation selection.

Removing selection reduces mean pLDDT by -32.26 points in the experimental phase ($72.89 \rightarrow 40.63$). It reduces mean pLDDT by -23.43 points on the whole PDB dataset ($71.49 \rightarrow 48.06$). It reduces mean pLDDT by -19.56 points on the SwissProt 106k corpus ($71.73 \rightarrow 52.17$).

In every phase, unfiltered outputs exhibit lower FD-seq (0.0768, 0.0413, 0.029 vs selected 0.2661, 0.1442, 0.065).

Selection concentrates candidate pools on high-likelihood modes. This moves the selected set away from full reference embedding breadth. Thus, FD-seq measures embedding coverage rather than foldability quality.

5.2 Analysis of Design Solutions

5.2.1 Encoder Capacity and Family

Varying encoder parameters by 19-fold (8M to 150M) changes mean pLDDT by only 1.62 points, from 71.83 to 73.45. The 8M encoder produced 1,533 sequences above pLDDT 70, compared with 1,580 for the 35M baseline. Increasing encoder capacity 4.3-fold (35M $\rightarrow$ 150M) yields a marginal $+0.56$ pLDDT gain and increases latent memory footprints by 33%. Representation capacity is not the binding constraint on foldability. However, ESM2-35M marginally outperformed in terms of diversity and also, generates identical novelty. Among the three options we considered for encoder, esm2 150M gives best pLDDT but substantially reduces novelty and diversity. A frozen encoder doesn't impact the architecture weight much, and to have a balanced approach towards pLDDT, we used esm2 35M as our primary encoder. The 420M Protbert encoder family resulted in the lowest mean pLDDT (68.26) and a 43.6% reduction in confident outputs (891 above 70). However, it achieved the lowest FD-seq in the phase (0.1355), illustrating that FD-seq can move opposite to structural foldability.

5.2.2 Training Duration

Doubling training duration (180 $\rightarrow$ 360 epochs) increased mean pLDDT by $+7.63$ points ($72.89 \rightarrow 80.52$). High-confidence outputs (> 80) increased from 493 to 1,202 ($+144\%$).

However, diversity degraded severely. CD@0.5 fell from 0.3129 to 0.1936 (-38%). Novelty (L1) fell from 0.3027 to 0.1895 (-37%). CD@0.95 dropped to 0.9955. FD-seq increased to 0.2921.

Longer training caused greater concentration, so we initialized on the primary target of ours which being mean pLDDT > 70 and considered the scores of 180 epoch runs

in PDB (2000-2015) and PDB (1971-2015)

However, for Swissprot dataset, as we got to epoch 180, then generated sequences, we got pLDDT mean 68.64, to achieve our target, we adjusted the epoch number to 225 and it generated sequences with mean pLDDT with 71.73 without lessening the novelty (decreased by 1%) or diversity (went from 0.9975 to 0.9920).

5.2.3 Noise Schedule

Replacing the tangent schedule with a cosine schedule reduced mean pLDDT from 72.89 to 54.02 (-18.87 points). The tangent schedule allocates 92 sampling steps to $t < 0.1$. The cosine schedule allocates only 4 steps. This leaves the model unable to resolve local residue identity.

5.2.4 Proxy Foldability Objective

The foldability proxy satisfied its hinge condition ($\tau = 93.0$) within four epochs. The loss reached 0.003. The objective remained inactive for the next 150 epochs. In another attempt with $\tau = 95.0$, it still converged just in 52 epochs, sequences were generated with this checkpoint and further analysis weren't performed on this proxy.

Downstream, it yielded a minor pLDDT gain ($+1.67$ to 74.56) with an extreme diversity penalty. CD@0.5 fell to 0.2415 from 0.3129. FD-seq rose to 0.3269.

5.2.5 Bio Composition Objectives

Differentiable composition losses produced a null result against the matched unselected control (40.69 vs 40.63 pLDDT). Composition statistics do not guarantee foldability.

Without a straight-through estimator, softmax composition scoring is degenerate. A uniform hedged decoder scores 0.92 3-mer plausibility, compared with 0.31 for real proteins.

5.3 Final Design Adjustments

Candidate filtering strategies were evaluated on the experimental phase. Global score ranking was compared with two cluster-aware variants using HDBSCAN (42 clusters + 2,726 noise points [27.3%]).

1. **Score-Scaled Retention:** Yielded 71.34 pLDDT and 0.3159 CD@0.5. Among the selected sequences, 1675 out of 2000 (roughly 80%) came from two clusters, thus didn't contribute to diversity, the cd 0.5 score remaining almost same (0.3129 and 0.3159) proves that. Thus, this implementation wasn't carried out for further runs.
2. **Uniform 20% Retention:** Preserved diversity (CD@0.5 0.8201, Novelty 0.6004) but caused pLDDT to collapse to 43.20, only +2.57 points above unfiltered generation.

Cluster-aware selection was removed because diversity is already addressed upstream through corpus deduplication ($CD@0.5 = 0.9975$ on SwissProt 106k without cluster filtering) and for PDB pipelines, the clustering layer didn't add enough balanced value, score scaled retention didn't improve diversity and uniform retention lowered pLDDT to 43.20.

5.4 Statistical Analysis

5.4.1 Experimental Phase (PDB 2000–2015)

Table 5.1 - summarises all measured configurations on the curated structural subset, anchored against the real sequence baseline (78.87 pLDDT).

Table 5.1: Statistical analysis of all measured configurations in the experimental phase (PDB 2000–2015).

Configuration / Ablation	pLDDT mean	FD-seq	Novelty (L1)	CD@0.5
Baseline (35M, tangent, 180 ep)	72.89	0.2661	0.3027	0.3129
8M encoder ($D = 320$)	71.83	0.2502	0.3090	0.3089
150M encoder ($D = 640$)	73.45	0.1539	0.2598	0.2706
Protbert encoder ($D = 1024$)	68.26	0.1462	0.2880	0.2920
270 epochs duration	78.61	0.3004	0.2119	0.2025
360 epochs duration	80.52	0.2921	0.1895	0.1936
Cosine noise schedule	54.02	0.1335	0.5251	1.0000
Proxy foldability objective	74.56	0.3269	0.2830	0.2415
Cluster selection (score-scaled)	71.34	0.2518	0.3106	0.3159
Cluster selection (uniform 20%)	43.20	0.0704	0.6004	0.8201
Bio Composition objectives	40.69	0.0841	0.5918	0.8216
No selection (unfiltered 2,000)	40.63	0.0768	0.5969	0.8307

5.4.2 Whole PDB Dataset (1971–2015)

Table 5.2 presents results for the whole structural archive across selected and unfiltered 180-epoch runs. The main difference being in this process 40-512 length sequences were selected instead of standard 128-256. The baseline of real sequence foldability of the dataset (78.87 pLDDT).

Table 5.2: Statistical analysis of the whole PDB dataset (1971–2015) configurations.

Configuration	pLDDT mean	FD-seq	Novelty (L1)	CD@0.5
Whole PDB, 180 ep (Selected)	71.49	0.1442	0.4296	0.6578
Whole PDB, 180 ep (Unfiltered)	48.06	0.0413	0.6325	0.9287

5.4.3 SwissProt 106k Corpus

Table 5.3 details performance on the redundancy-controlled corpus, anchored against the real dataset foldability reference (80.7 pLDDT).

Table 5.3: Statistical analysis of the SwissProt 106k corpus configurations.

Configuration	pLDDT mean	FD-seq	Novelty (L1)	CD@0.5
SwissProt, 135 ep (Selected)	66.67	0.0521	0.5906	1.0000
SwissProt, 180 ep (Selected)	68.64	0.0600	0.5633	0.9975
Swissprot 225 ep (Selected)	71.73	0.065	0.5425	0.9920
SwissProt, 180 ep (Unfiltered)	50.35	0.0361	0.6633	1.0000
Swissprot,225 ep (Unfiltered)	52.17	0.029	0.6533	1.0000

5.4.4 Master Consolidated Analysis and Scaling Relationships

Evaluating selection efficiency—pLDDT points gained per percentage point of novelty surrendered—reveals that selection is $1.61\times$ more efficient on deduplicated data:

- **Experimental Phase:** $32.26 \text{ pLDDT} \div 29.42 \text{ novelty pp} = 1.10 \text{ pLDDT/pp}$
- **Whole PDB Dataset:** $23.43 \text{ pLDDT} \div 20.29 \text{ novelty pp} = 1.15 \text{ pLDDT/pp}$
- **SwissProt 106k Corpus:** $19.56 \text{ pLDDT} \div 11.08 \text{ novelty pp} = 1.77 \text{ pLDDT/pp}$

Moving across regimes (Experimental Phase $\rightarrow$ SwissProt 106k) costs 1.16 pLDDT points ($72.89 \rightarrow 71.73$) while gaining $+0.6782$ CD@0.5, $+0.2398$ Novelty (L1), and a 4.09-fold FD-seq reduction. FD-seq decreases log-linearly with corpus size ($0.2661 \rightarrow 0.1442 \rightarrow 0.0650$), where a $5.78\times$ corpus growth produces a $4.09\times$ reduction in embedding distance.

5.5 Comparisons and Relationships

The proposed framework was benchmarked against 10 published sequence generation models, as presented in Table 5.4. Cross-study FD-seq comparison is invalid due to $L2$ -normalisation scale factors ($29.1\times-33.2\times$) and is omitted.

Table 5.4: Cross-model benchmark comparison against published sequence generation baselines.

Dataset	pLDDT mean	CD@0.5	Novelty (L1)
Dataset	80.7	1.000	25.3
Random sequences	24.8	1.000	85.1
Model/Baseline	pLDDT mean	CD@0.5	Novelty (L1)
Walk-Jump	32.4	1.000	82.2
RITA	43.9	0.988	60.4
proteinGAN	30.4	0.955	83.5
SeqDesign	43.1	0.929	81.2
EvoDiff-OADM	37.1	0.986	77.6
D3PM	36.7	0.994	78.4
DFM	37.8	0.996	77.2
DPLM	84.0	0.494	11.5
nanoGPT	61.0	0.900	53.7
DiMA	83.3	0.611	35.7
Our Framework (Selected)	71.73	0.9920	54.2
Our Framework (Unfiltered)	52.17	1.0000	65.3

Our framework ranks third overall in structural foldability (71.73 pLDDT), outperforming autoregressive, GAN, score-based, and discrete-diffusion baselines by 10.73–41.33 pLDDT points.

Even unfiltered outputs (52.17 pLDDT) remain superior to 7 of 10 baselines without post-generation selection, in terms of foldability

Among systems generating structurally plausible output (pLDDT > 60), our framework achieves the highest sequence diversity (CD@0.5 = 0.9920). Leading quality systems have lower diversity (DPLM 0.494, DiMA 0.611) and Novelty (DiMA-35.7, DPLM 11.5)

Systems with near-perfect diversity (Walk-Jump 1.000, DFM 0.996) have low foldability, with pLDDT ≤ 37.8 .

5.6 Discussions

5.6.1 Data Redundancy and Diversity Collapse

The absence of diversity in the structural training runs is a characteristic of the corpus and not of the model. The observed internal repetitiveness of a structural set is $\text{Rep}(\text{train}) = 0.7466$, which indicates that many of the n -mers in one training sequence are also found in other sequences. A generative model that fits such a corpus produces the same redundancy and the tight distribution of outputs is a correct one, not a failure. It is evident from the scaling of the corpus without altering its composition. If the number of training sequences is increased from 18,349 to 51,829, $\text{Rep}(\text{train})$ is essentially unchanged (0.7453). The redundancy was carried along with the data: If diversity losses were due to inadequate data, then more sequences would have alleviated the losses, and if they were due to limited capacity of the model, then they would be due to the same, since the diversity losses would not have been resolved by obtaining extra sequences. When redundant sequences are removed from the sequence space, however, $\text{Rep}(\text{train})$ is reduced to 0.0873 (an order of magnitude) and generated cluster diversity at $\text{CD}@0.5$ is increased to 0.9920. With the same architecture trained for the same number of epochs, one shifts from close to collapse to a near separation between the two only on the basis of what it was taught. The bottom line is that redundancy control is not in the objective, it's in the data pipeline. It would be a symptom of the corpus being treated with a property of the model to try to recover diversity by regularisation or sampling the temperature; the measurements here indicate that the effort is better directed at deduplication.

5.6.2 Mechanics of Proxy Objectives

The auxiliary objectives were added to provide a training signal which could not be obtained by reconstruction: a sense of where in the latent space foldable proteins reside. In actuality, these are not actions that they perform based on what they measure, but are actions that they perform that are relatively easy to satisfy. This is clearly seen in the proxy foldability term. The hinge was cleared at epoch 34 for target $\tau = 93.0$, and at epoch 52 for target $\tau = 95.0$. The term is one-sided, which means that once the threshold is passed, the gradient is completely removed and from that point onwards, the term doesn't add to a training process. The other epochs were then only periods of reconstruction, and this explains why the two threshold settings did not give very different results. Having a proxy that a model can satisfy early is not a constraint on the model for most of its training, it's just a nudge at the beginning and its apparent effect on the final result is less than the loss curves imply. The composition goals did not succeed in the same but different manner. They are degenerate when evaluated on the raw softmax distribution, with a 3-mer score of 0.92 when the decoder distributes the probability mass evenly across the residues and never makes any sequence commitments. The result is that it is possible to reduce the objective by becoming less true than it is intended to be, which is the opposite of the intended aim. This is addressed by straight-through discretisation wherein the statistics are calculated on the actual discrete sequence the decoder is expected to output during testing, and the training-time and post-hoc scores match within 8.1×10^{-8} . But the hole was not closed by any measurable

increase in the foldability at the downstream end. These statistics are just fine as filters applied after generation, but insufficiently correlated to structure to be used as training targets.

5.6.3 System Bottlenecks and Headroom

Two components, both sharing the same training objective, need to be separated in order to locate the bottleneck. The decoder is very accurate on clean latents with a cross-entropy of 0.0001 and 100.0% reconstruction accuracy on the curated structural corpus. If the latent is correct, then the sequence of residues can be recovered and the job is done. For latents predicted by the denoiser, the same decoder decodes the latents at 43.4% to 47.2%. The gap is not the decoder’s characteristic but a measure of how far the denoiser’s output is from the true latent. The latent-space measurement concurs. A denoiser which produces no signal whatsoever would have an MSE of 0.91 on normalised targets, whereas trained denoisers have an MSE of 0.63–0.69, which corresponds to the recovery of about a quarter to a third of the signal. This is a significant departure from the null model, but most of the latent remains unexplained, and it is this residual error that the decoder is required to accommodate. Capacity to denoise, not to decode, is the limiting factor, across the structural runs. One number throws the other numbers into a shade of confusion, but one that is informative. The decoder’s accuracy on clean latents did not reach 100%, but rather reached a plateau at 61.2% on the SwissProt 106k corpus regardless of learning-rate variation, which ranges over 2 orders of magnitude. A most likely interpretation is that the “near-perfect” accuracy on the structural corpus was a lot due to its redundancy, and that the decoder would be able to be only partially “memorising” on such a corpus with many near-duplicate sequences, whereas on a deduplicated corpus an order of magnitude larger, this would not be the case. Whether this 61.2% is the true maximum capacity for a 3-layer decoder of this width, or it’s just because the training budget ran out in the experiments is yet to be determined and represents the most direct source of headroom the framework has.

5.6.4 Methodological Limitations

The results given above are subject to several limitations which are noted below to ensure that the figures are interpreted at the level of confidence for which they are valid.

1. **Homology leakage in temporal splits:** The deposition-year does not separate the protein families of the structural corpus. In a held-out set, a sequence of entries that are close to each other may be on separate sides of the boundary in different years of the held-out set. Novelty L2 is therefore a positive result when assessed against this kind of a split and should be interpreted with caution as an example of generalisation.
2. **Confounded phase parameters:** Both structural phases change at more than one point: length band (128–256 to 40–512 residues) and deposition-year window (two different windows for each phase). Any deviation between them

will be a result of the joint effect of these two changes and the contribution of either change cannot be separated from these runs.

3. **Proxy structural confidence:** All of the confidence numbers quoted are pLDDT values; that is, they represent the level of confidence that a structure predictor has in its own prediction. It is not a measurement of folding of the protein. No sequence has been synthesised or experimentally characterised so the confidence numbers are to be treated as a computational screen and not as a validation.
4. **Within-run score normalisation:** Selection scores are min–max normalized within the generated pools and so are valid for comparison within a run but not between runs. The composite score of one run (0.8) is not necessarily equal to the composite score of another run (0.8) and so there is no fixed threshold that can be carried from one run to another.
5. **Clustering instability:** HDBSCAN is based on density and the structure of density of the candidate pool is related to the embedding space in which it is measured. Cluster counts and assignments did not impact the results in desired way, which is why the clustering stage excluded from the reported pipelines.
6. **FD-seq scale:** Here it is calculated with L2-normalised mean embedding. This results in the metric being internally consistent and being comparable to the values of other configurations of this work, but values are on a different scale than published values based on raw embedding calculations. It is not meaningful to compare FD-seq numbers across papers unless they are computed on a common basis.

Chapter 6

Conclusion

6.1 Summary of Key Findings

This dissertation establishes a working foundation for budget-constrained protein sequence design using latent diffusion. The study systematically examines architectural, data, and sampling dimensions through ablation experiments. These experiments lead to a set of empirical findings that share a common theme. The factors controlling output quality are not always those predicted by scaling intuition.

Foldable generation is achievable on a single consumer GPU. The complete pipeline trains, generates, and evaluates on one NVIDIA RTX 4090 with 24 GB of VRAM. It achieves a mean structural confidence of pLDDT 72.89 on the curated structural corpus. The corresponding value for real training sequences is 78.87 under the same structure predictor.

On the SwissProt corpus used for cross-system comparison, the selected output ranks third into ten published generative frameworks in foldability. It exceeds the reported results of autoregressive, GAN, score-based, and discrete-diffusion baselines. Several of these frameworks were trained on multi-node industrial clusters. This comparison is indicative rather than controlled. The baseline figures come from the authors' own evaluation corpora. Nevertheless, the framework reaches this position with a fraction of the computational resources.

Encoder capacity is not the main constraint on foldability. Varying the frozen encoder from 8M to 150M parameters changed mean pLDDT by only 1.62 points. The 420M encoder from another model family (prot-bert) achieved a lower mean pLDDT of 68.26. Only 891 outputs exceeded pLDDT 70, representing a 41% reduction in confident outputs. These results suggest that encoder size alone does not determine foldability. Small encoders can support continuous latent diffusion while reducing computational costs.

Noise schedule geometry is decisive. Replacing the hyperbolic tangent schedule with a standard cosine schedule reduced mean pLDDT by 18.87 points. This was the largest single-factor loss measured in this work.

The mechanism is step allocation rather than the schedule family itself. Under

signal-uniform sampling, the tangent schedule allocates 92 of 225 steps to the low-noise regime below $t = 0.1$. This is where residue identity is resolved. The cosine schedule allocates only 4 of 250 steps to this regime. The cosine configuration produced 1,288 sequences above pLDDT 50, but only 80 above 70 and none above 80. This distribution suggests that the sequences acquire global shape but fail to resolve local detail.

Training duration is the strongest positive architectural lever, and the most expensive in diversity. Doubling the diffusion training budget from 180 to 360 epochs increased mean pLDDT from 72.89 to 80.52, a gain of 7.63 points. The number of sequences above pLDDT 80 increased from 493 to 1,202 (+144%). However, diversity declined. CD@0.5 fell from 0.3129 to 0.1936 (−38%). Novelty against the training set fell by 37%. This is the clearest quality–diversity trade-off measured in this work. It shows why training duration cannot be extended indefinitely in pursuit of higher structural confidence.

Post-generation selection contributes more than any architectural choice available at this budget. Removing the selection stage costs 32.26 pLDDT points on the curated structural corpus, 23.43 on the whole structural archive, and 19.56 on the deduplicated sequence corpus. In the first case, this is roughly four times the gain from doubling the training budget.

Selection is also more efficient on deduplicated data. It purchases 1.77 pLDDT points per percentage point of novelty surrendered, compared with 1.10 on the redundant corpus. Redundancy makes “folds well” and “resembles the training set” nearly the same property. A likelihood-based ranking cannot separate them. Deduplication separates these properties and allows selection to improve confidence at a lower novelty cost.

Corpus redundancy, not model capacity, governs generated diversity. Corpus repetition of 0.7466 and 0.7453 on the two structural corpora produced generated cluster diversity of 0.3129 and 0.6578, respectively. Reducing repetition to 0.0873 produced 0.9920 while preserving structural foldability. A data control reinforces this point. Tripling the corpus from 18,349 to 51,829 sequences left repetition unchanged at the third decimal place in the training set, but still increased generated sequence diversity.

The per-scale breakdown shows redundancy at every n -mer length up to 64. This indicates recurring whole domains and sequences rather than common localised motifs. The comparison has no obvious confound. There is no obvious confusion in the comparison. The curated and deduplicated corpora have nearly comparable length profiles and also share a length band. They differ materially in redundancy.

Both auxiliary training objectives failed, for different and instructive reasons. The learned foldability proxy used a one-sided hinge against a frozen confidence predictor. It reached its target at epoch 52. It then contributed approximately 0.003 loss for the remaining 130 epochs.

Across the full run, it gained 1.67 pLDDT points. However, cluster diversity fell by 0.0714, from 0.3129 to 0.2415. This was a worse trade-off than simply training longer.

Three differentiable composition objectives were used. They included a k -mer divergence, a polar fraction band, and a composition entropy floor. They were hinged at empirical quantiles of the real distribution. Thus, 90%, 50%, and 90% of real training sequences remained in violation. The gradient therefore did not saturate.

The objectives were evaluated on discrete sequences through a straight-through estimator. This prevented hedged probability distributions from satisfying the objectives cheaply. Despite these precautions, the objectives produced a null result against the matched unselected control. Mean pLDDT was 40.69 versus 40.63.

This result is informative. The objectives were implemented but still failed to improve foldability. This suggests that the binding constraint lies beyond amino acid composition and 3-mer statistics.

Cluster-aware candidate selection is unnecessary. Three retention rules were evaluated. Scaling retention by cluster quality reproduced global ranking almost exactly. Two of 42 clusters supplied 84% of the final pool.

Uniform retention preserved diversity but reduced quality to 43.20 pLDDT. This was only 2.57 points above the unselected generation.

Diversity is already addressed upstream through corpus deduplication. Therefore, cluster-aware selection adds computational overhead without measurable structural benefit. It was removed from the final design.

The denoiser, rather than the decoder or encoder, is the main system bottleneck. Realistic comparison of sequence reconstruction of true and predicted latents isolates information that is lost. The decoder reconstructs the residue from the ground-truth eliminate any and all latents accurately. Recovers approximately to 95% at the noise level it is subjected to encounters near the end of sampling. This falls to 43.4–47.2% across the full noise range, including regimes never used during inference. Latent mean squared error settles at 0.63–0.69, compared with 0.91 for a model that emits constant zeros. Thus, the denoiser recovers roughly a quarter to a third of the available signal. This finding directly explains the encoder result. The representation is not the main limitation, and neither is the discrete decoding step.

Finally, the distributional metric used throughout the literature measures coverage rather than quality. The lowest FD-seq distances are found in unfiltered pools and in the worst-performing configurations. The highest distance belongs to the configuration with the best mean pLDDT in its phase. Selection usually lessens the pool to folded towards sequence which should show better foldability when pLDDT is measured. This raises FD-seq while also improving quality. By itself, the metric is not a reliable quality measure. Alongside cluster diversity, it helps distinguish between matching a distribution and matching its dominant modes.

6.2 Contributions to the Field

The contributions of this work are stated according to the strength of the evidence. The thesis reimplements and extends an existing latent-diffusion approach rather than proposing a new generative paradigm. Its value lies in what it establishes about operating this class of model under constraints.

The principal contribution is a **characterised compute-constrained design point**. The framework trains end to end, generates sequences, and performs evaluation on a single 24 GB consumer GPU. The work also provides the full ablation record of what matters at this budget. It therefore provides a practical map for researchers with similar hardware. It shows which levers to change, which to leave alone, and in what order. The work also provides methodological evidence that model scale is not a principal limitation on generative protein frameworks. This lowers the barrier to entry for resource-constrained academic laboratories.

The work also contributes a **joint position on quality and diversity**. Among published systems with pLDDT above 70, this framework achieves the highest measured sequence diversity. It reaches $CD@0.5 = 0.9920$ with a mean pLDDT of 71.73. The two systems with higher foldability have roughly half the diversity of this framework. Every system with near-perfect diversity has pLDDT at or below 37.8.

A further contribution is the finding that **corpus redundancy and corpus size, rather than model scale, govern generated diversity**. This finding is established across three corpora. The comparison is free of length confounds and is quantified on both sides. The three training sets produce markedly different diversity in their generated outputs. This reflects differences in the quality and quantity of their data. The finding reframes an observed failure mode of generative protein models as a data-curation problem with a comparatively cheap solution.

Finally, as a minor contribution, the thesis offers a **detailed analysis of auxiliary objective failure mechanics** and openly reports negative results. Two distinct failure modes are identified and separated. The first is saturation. It occurs when an objective is hinged against a threshold that the model reaches early. The objective then ceases to provide a training signal. The second is degeneracy in statistics computed on hedged probability distributions. A maximally uncertain decoder scores 0.92 on 3-mer plausibility, compared with 0.31 for real proteins. Thus, the objective can be optimised by a model that has learned nothing.

Alongside these, the thesis reports two failed training objectives, one selection stage that was built and later removed, and one schedule variant that cost 18.87 pLDDT points. It also reports a bottleneck analysis that contradicted the intuition behind the encoder ablations. These results are reported with their mechanisms rather than omitted.

6.3 Future Directions

The recommendations that follow are ordered according to the bottleneck analysis. The denoiser recovers roughly a third of the available latent signal. The decoder reads clean latents without error, however on the SwissPort run it converges at 61.2% . Therefore, work on the denoiser has the highest expected return. Work on the encoder has the lowest.

Among high-yield technical initiatives, the most direct option is to increase denoiser capacity and training duration. Increasing the denoiser from six to twelve layers would double its depth. Its size would increase from approximately 22M to 45M parameters. This remains within the memory limit. The 360-epoch result already shows a gain of 7.63 pLDDT points. It remains unknown whether greater denoiser capacity would add to this gain.

Second, the noise schedule warrants a focused sweep. The schedule cost 18.87 pLDDT points in one comparison. However, only one setting of its steepness parameter was tested. A one-dimensional sweep from $d = 5$ to $d = 15$ is among the cheapest available experiments. It also targets the most sensitive factor identified in this work. The step-allocation analysis suggests that the number of sampling steps below $t = 0.1$ should be optimised rather than the schedule family itself.

Third, the structural corpora should be deduplicated. The central diversity finding compares a redundant structural corpus with a deduplicated sequence corpus. The benefit has therefore not been demonstrated on structural data. Structural alignment clustering using TM-align or Foldseek with a TM-score threshold above 0.6 could test whether the deduplicated regime is reachable on structural data. It would also remove the remaining confound in the argument. This requires no retraining infrastructure.

Among methodological verification initiatives, multi-seed replication is the most consequential. Every configuration reported here was run once. This gives a claim band of roughly 2 pLDDT points. Replicating the baseline and contested comparisons across 10 seeds would reduce this uncertainty and strengthen the other claims.

Second, identity-disjoint evaluation should be performed. Temporal splits leave homologues on both sides of the boundary. This is why the held-out novelty statistic is not reported as a headline figure. Replacing the year-based split with a cluster-disjoint split below 20% sequence identity would put the structural corpora on the same footing as the sequence corpus.

Third, orthogonal validation should be pursued. This is the most valuable option without using wet-lab work. The structure predictor may share biases with the language model used for selection. Evaluating the same generated pools with an independent structure predictor or a multiple-sequence-alignment-based method would help determine whether the reported quality comes from the sequences or from one evaluator.

Among open technical frontiers, we found that, correct amino acid statistics do not produce foldable sequences. Which is explained by how lowest FD-seq value generations usually gave bad foldability. A remaining possibility is a differentiable signal that carries structural information. Examples include predicted contact or distance maps. Such objectives should follow two design rules established by this work. They should be evaluated on discrete sequences using a straight-through estimator. They should also be hinged against an empirical level that the model does not reach early. The proxy-objective failure shows that the second rule is important.

A second frontier is a trained selection ranker. The current composite score achieves a rank correlation with true pLDDT of at most 0.624. This falls to 0.448 on the deduplicated data. A ranker could be trained directly on the pLDDT labels already produced by the folding runs. Several thousand such labels now exist per corpus. This would require no additional labelling effort.

Two directions are not recommended based on the evidence presented here. Scaling the frozen encoder beyond 150M is poorly motivated. Scaling from 8M to 150M changed mean pLDDT by only 1.62 points. The largest encoder family also performed worst in the separate 420M comparison. Larger encoders also impose greater memory and preprocessing costs.

Cluster-aware candidate selection should also not be revisited in the form tested here. If output diversity proves unsatisfactory, the corpus is the place to intervene. This requires less complexity and does not reduce structural quality.

Bibliography

- [1] Anonymous, *Leveraging deep generative models for computational protein design and optimization*, 2024. arXiv: 2408.17241 [q-bio.QM]. [Online]. Available: <https://arxiv.org/abs/2408.17241>
- [2] A. J. Riesselman, J. B. Ingraham, and D. S. Marks, “Deep generative models of genetic variation capture mutation effects,” *Nature Methods*, vol. 15, no. 10, pp. 816–822, 2018. DOI: 10.1038/s41592-018-0138-4
- [3] A. Hawkins-Hooker, F. Depardieu, S. Baur, G. Couairon, A. Chen, and D. Bikard, “Generating functional protein variants with variational autoencoders,” *PLoS Computational Biology*, vol. 17, no. 2, e1008736, 2021. DOI: 10.1371/journal.pcbi.1008736 [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7946179/>
- [4] S. Lyu, S. Sowlati Hashjin, and M. Garton, *Proteinvaе: Variational autoencoder for translational protein design*, 2023. [Online]. Available: <https://www.biorxiv.org/content/10.1101/2023.03.04.531110v1>
- [5] M. Mardikoraem, Z. Wang, N. Pascual, and D. Woldring, *Generative models for protein sequence modeling: Recent advances and future directions*, 2023. [Online]. Available: <https://academic.oup.com/bib/article/24/6/bbad358/7325909>
- [6] N. Anand and T. Achim, *Protein structure and sequence generation with equivariant denoising diffusion probabilistic models*, 2022. arXiv: 2205.15019 [q-bio.QM]. [Online]. Available: <https://arxiv.org/abs/2205.15019>
- [7] L. Zongying et al., *Taxdiff: Taxonomic-guided diffusion model for protein sequence generation*, 2024. arXiv: 2402.17156 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2402.17156>
- [8] J. S. Lee, J. Kim, and P. M. Kim, “Score-based generative modeling for de novo protein design,” *Nature Computational Science*, vol. 3, no. 5, pp. 382–392, 2023. DOI: 10.1038/s43588-023-00440-3
- [9] J. L. Watson, D. Juergens, N. R. Bennett, B. L. Trippe, J. Yim, H. E. Eisenach, et al., “De novo design of protein structure and function with rfdiffusion,” *Nature*, vol. 620, no. 7976, pp. 1089–1100, 2023. DOI: 10.1038/s41586-023-06415-8
- [10] J. B. Ingraham, M. Baranov, Z. Costello, et al., “Illuminating protein space with a programmable generative model,” *Nature*, vol. 623, no. 7989, pp. 1070–1078, 2023. DOI: 10.1038/s41586-023-06728-8

- [11] V. Meshchaninov et al., *Diffusion on language model encodings for protein sequence generation*, 2025. [Online]. Available: <https://openreview.net/forum?id=LoXJlAW3gU>
- [12] E. Quinn, G. Jebali, M. Seince, and O. Bent, *Discriminative protein sequence modelling with latent space diffusion*, 2025. arXiv: 2503.18551 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2503.18551>
- [13] L. Hallee, N. Rafailidis, D. B. Bichara, and J. P. Gleghorn, *Diffusion sequence models for enhanced protein representation and generation*, 2025. arXiv: 2506.08293 [q-bio.BM]. [Online]. Available: <https://arxiv.org/abs/2506.08293>
- [14] S. L. Lisanza, J. M. Gershon, S. W. K. Tipps, J. N. Sims, L. Arnoldt, S. J. Hendel, et al., “Multistate and functional protein design using rosettafold sequence space diffusion,” *Nature Biotechnology*, vol. 43, no. 8, pp. 1288–1298, 2025. DOI: 10.1038/s41587-024-02395-w
- [15] N. Ferruz, S. Schmidt, and B. Höcker, “Protgpt2 is a deep unsupervised language model for protein design,” *Nature Communications*, vol. 13, no. 1, p. 4348, 2022. DOI: 10.1038/s41467-022-32007-7
- [16] A. Madani, B. Krause, E. R. Greene, et al., “Large language models generate functional protein sequences across diverse families,” *Nature Biotechnology*, vol. 41, pp. 1099–1106, 2023. DOI: 10.1038/s41587-022-01618-2
- [17] Z. Lin, H. Akin, et al., “Evolutionary-scale prediction of atomic-level protein structure with a language model,” *Science*, vol. 379, no. 6637, pp. 1123–1130, 2023. DOI: 10.1126/science.ade2574
- [18] W. Liu, Z. Wang, R. You, et al., “Plmsearch: Protein language model powers accurate and fast sequence search for remote homology,” *Nature Communications*, vol. 15, no. 1, p. 2775, 2024. DOI: 10.1038/s41467-024-46808-5
- [19] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990. DOI: 10.1016/S0022-2836(05)80360-2
- [20] J. Dauparas, I. Anishchenko, N. Bennett, H. Bai, R. J. Ragotte, L. F. Milles, et al., “Robust deep learning-based protein sequence design using proteinmpnn,” *Science*, vol. 378, no. 6615, pp. 49–56, 2022. DOI: 10.1126/science.add2187
- [21] L. F. Krapp et al., “Context-aware geometric deep learning for protein sequence design,” *Nature Communications*, vol. 15, no. 1, p. 6273, 2024. DOI: 10.1038/s41467-024-50571-y
- [22] S. Liu et al., “A text-guided protein design framework,” *Nature Machine Intelligence*, vol. 7, no. 4, pp. 580–591, 2025. DOI: 10.1038/s42256-025-01011-z
- [23] C. Ren, D. He, and J. Huang, “De novo functional protein sequence generation: Overcoming data scarcity through regeneration and large language models,” *Briefings in Bioinformatics*, vol. 27, no. 2, bbag095, 2026. DOI: 10.1093/bib/bbag095
- [24] P. Bai et al., “Mask-prior-guided denoising diffusion improves inverse protein folding,” *Nature Machine Intelligence*, vol. 7, no. 6, pp. 876–888, 2025. DOI: 10.1038/s42256-025-01042-6

- [25] Z. Wu, K. E. Johnston, F. H. Arnold, and K. K. Yang, *Protein sequence design with deep generative models*, 2021. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/34051682/>
- [26] A. Strokach and P. M. Kim, *Deep generative modeling for protein design*, 2021. [Online]. Available: <https://arxiv.org/abs/2109.13754>
- [27] J. Trinquier, G. Uguzzoni, A. Pagnani, F. Zamponi, and M. Weigt, *Efficient generative modeling of protein sequences using simple autoregressive models*, 2021. [Online]. Available: <https://doi.org/10.1038/s41467-021-25756-4>
- [28] F. Wan, D. Kontogiorgos-Heintz, and C. de la Fuente-Núñez, *Deep generative models for peptide design*, 2022. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9189861/>
- [29] Anonymous, *Searching for protein variants with desired properties using deep generative models*, 2021. [Online]. Available: <https://www.bohrium.com/paper-details/searching-for-protein-variants-with-desired-properties-using-deep-generative-models/890208541254615835-9382>
- [30] Z. Wu, J. Zou, A. Boyd, K. K. Yang, F. H. Arnold, et al., *Protein sequence design with deep generative models: Perspectives from structure prediction and protein language modeling*, 2023. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10599991/>
- [31] D. S. Berman, C. Howser, T. Mehoke, and J. D. Evans, *Mutagan: A seq2seq gan framework to predict mutations of evolving protein populations*, 2020. arXiv: 2008.11790 [q-bio.QM]. [Online]. Available: <https://arxiv.org/abs/2008.11790>
- [32] J. Frazer et al., *Disease variant prediction with deep generative models of evolutionary data*, 2021. [Online]. Available: <https://www.nature.com/articles/s41586-021-04043-8>
- [33] E. Hoogeboom, T. Cohen, and J. M. Tomczak, *Argmax flows and multinomial diffusion: Learning categorical distributions*, 2021. arXiv: 2102.05379 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2102.05379>
- [34] S. Gu et al., *Vector quantized diffusion model for text-to-image synthesis*, 2022. arXiv: 2111.14822 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2111.14822>
- [35] A. Nguyen et al., *Latent consistency-matching diffusion for vector-quantized models*, 2022. [Online]. Available: <https://openreview.net/forum?id=um0k1wkLPu>
- [36] J. Phillips et al., *Generating immune-aware sars-cov-2 spike proteins for universal vaccine design*, 2022. [Online]. Available: <https://proceedings.mlr.press/v184/phillips22a.html>
- [37] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, *High-resolution image synthesis with latent diffusion models*, 2022. arXiv: 2112.10752 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2112.10752>
- [38] Y. Chen et al., “Deep generative model for drug design from protein target sequence,” *Journal of Cheminformatics*, vol. 15, no. 1, p. 38, 2023. DOI: 10.1186/s13321-023-00702-2 [Online]. Available: <https://doi.org/10.1186/s13321-023-00702-2>

- [39] N. Gruver et al., *Protein design with guided discrete diffusion*, 2023. arXiv: 2305.20009 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2305.20009>
- [40] K. Wu, K. K. Yang, et al., *Predicting the antigenic evolution of sars-cov-2 with deep learning*, 2023. [Online]. Available: <https://www.nature.com/articles/s41467-023-39199-6>
- [41] E. Ma et al., “A predictive language model for sars-cov-2 evolution,” *Signal Transduction and Targeted Therapy*, vol. 9, no. 1, p. 353, 2024. DOI: 10.1038/s41392-024-02066-x [Online]. Available: <https://doi.org/10.1038/s41392-024-02066-x>
- [42] S. Saqib, Z. Mumtaz, H. Ahmed, A. Ali, O. Qazi, and M. Z. Yousaf, *Generative deep neural networks for estimating hypervariability in hepatitis b and c virus genomes*, 2024. DOI: 10.21203/rs.3.rs-5560102/v1
- [43] D. Hamelin et al., *Predicting pathogen evolution and immune evasion in the age of artificial intelligence*, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2001037025001138>
- [44] X. Liu et al., *Generative prediction of real-world prevalent sars-cov-2 mutation with in silico virus evolution*, 2025. [Online]. Available: <https://academic.oup.com/bib/article/26/3/bbaf276/8168831>
- [45] J. Shi, K. Han, Z. Wang, A. Doucet, and M. K. Titsias, *Simplified and generalized masked diffusion for discrete data*, 2025. arXiv: 2406.04329 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2406.04329>
- [46] G. Terai and K. Asai, “Deep generative model of rnas based on variational autoencoder with context-free grammar,” *Bioinformatics*, vol. 41, no. 8, btaf427, Jul. 2025. DOI: 10.1093/bioinformatics/btaf427 [Online]. Available: <https://doi.org/10.1093/bioinformatics/btaf427>
- [47] Y. Wang and K. Cho, “Enhancing car-t cell activity prediction via fine-tuning protein language models with generated car sequences,” *BMC Bioinformatics*, 2024. [Online]. Available: <https://link.springer.com/article/10.1186/s12859-026-06401-7>
- [48] P. Kantroo, G. P. Wagner, and B. B. Machta, “Pseudo-perplexity in one fell swoop for protein fitness estimation,” *PRX Life*, vol. 3, no. 3, p. 033 014, 2025. DOI: 10.1103/zhx7-hcmm